

Software Design Description

Francanglais Compiler and Dataset Platform

Source-grounded Architecture and Explained UML

Group members and matricules:

Kwete Ngouba Junior Rayan — ICTU20241377

Djemtchimo Noukui Bruno Jonatan — ICTU20241585

Name 3: _____ Matricule: _____

Version 1.3 — 19 September 2026

Engineering evidence, not a substitute for genuine fieldwork.

Revision basis: current working source based on 160945c, including the separate reference dictionary integration. The two supplied names/matricules are preserved; remaining identity details and contributions are not invented.

Reading design: A4 explanations are followed by 29 full-size UML sheets. Click a figure reference to inspect its sheet. The sheets retain readable label sizes rather than shrinking complex diagrams to tiny A4 text. Use actual-size or tiled printing for large sheets; the SRS remains A4.

Contents

1	Design Basis and UML Conventions	5
1.1	Implemented system and scope	5
1.2	How to read the models	5
2	Architecture and Deployment	5
2.1	Component dependencies	5
2.2	Deployment topology	6
3	Class Structure and Relationships	6
3.1	Desktop classes, optional devices and explicit errors	6
3.2	Compiler objects versus function-based algorithms	6
3.3	API request inheritance	7
3.4	Public analysis and translation results	7
3.5	Collection DTOs and mutation errors	7
3.6	Import candidates and coursework models	7
3.7	Provider wire-response classes	8
3.8	Provider service, settings and local grounding	8
3.9	Frontend runtime class and functional modules	8
4	Data Design, Transactions and Review Rules	8
4.1	Storage schema and compatibility	8
4.2	Transactional and identity boundaries	9
4.3	Reference vocabulary is not a research corpus	9
4.4	Audio and review ownership	9
5	Use Cases, Activity and State Models	9
5.1	User goals and optional extensions	9
5.2	One collection/save activity	10
5.3	Logical entry and selected-editor states	10
5.4	Audio readiness and retry states	10
6	Sequence Views: Detailed Interaction Explanations	10
6.1	Attachment and collection commit	10
6.2	Search and cached selection	11
6.3	Changed-field editing	11
6.4	Confirmed deletion	11
6.5	Recording, callback delivery and publication	11
6.6	Playback and external fallback	11
6.7	Statistics from one snapshot	12
6.8	Shared CSV transaction	12
6.9	Local-source translation	12
6.10	Chat history and provider boundaries	12
6.11	Read-only dictionary lookup and handoff	13
6.12	Import preview, suggestions and explicit saving	13
6.13	Grammar analysis and predictive parsing	13
6.14	Saved coursework snapshot and export	13
7	HTTP Interfaces and Failure Contracts	14
7.1	Complete application route inventory	14
7.2	Validation, errors and bounded work	14

8	Verification, Recovery and Human Acceptance	15
8.1	Recorded engineering checks	15
8.2	Reproducible commands and CI boundary	15
8.3	Backup, verify and restore procedure	16
8.4	Remaining human and operational gates	16
A	Full-size UML Sheets	17

List of Figures

1	Implemented component dependencies	18
2	Trusted local deployment and optional services	19
3	Desktop classes and audio ownership	20
4	Compiler classes and functional algorithms	21
5	API request hierarchy and chat cardinalities	22
6	Translation and analysis result relationships	23
7	Collection DTOs and mutation errors	24
8	Import and coursework class relationships	25
9	Provider wire-response hierarchy	26
10	Provider service, settings and grounding values	27
11	Frontend runtime class and functional modules	28
12	User goals and guarded optional extensions	29
13	Collection activity and failure decisions	30
14	Logical entry and selected-editor states	31
15	Audio readiness and retry states	32
16	Attachment and collection commit sequence	33
17	Search and cached selection sequence	34
18	Changed-field edit sequence	35
19	Confirmed deletion sequence	36
20	Recording, callbacks and WAV publication	37
21	Playback and OS fallback sequence	38
22	Single-snapshot statistics sequence	39
23	Shared locked CSV transaction sequence	40
24	Dataset-first translation sequence	41
25	Explicit chat and provider handling sequence	42
26	Preview, suggestion and reviewed import sequence	43
27	Grammar analysis and predictive parsing sequence	44
28	Saved coursework and draft export sequence	45
29	Separate reference lookup and explicit draft handoff	46

1 Design Basis and UML Conventions

1.1 Implemented system and scope

The source now contains the editable React frontend, FastAPI backend, deterministic lexer/parser and CustomTkinter collector. They are implemented, not proposed components. This document supersedes the old desktop-only SDD. The local CSV is shared; there is no database server, trained predictor, authenticated user model or public multi-tenant deployment.

Nine class views collectively cover **49 authored production classes**: 47 Python class declarations, the factory-generated lexer `Token`, and the TypeScript `ApiError`. The machine-readable mapping is `docs/diagrams/class-coverage.json`. Test helpers, maintenance tools, framework classes and erased TypeScript interfaces are outside that count. The remaining views are 14 sequences, two states, one activity, one use case, one component view and one deployment view.

1.2 How to read the models

- A hollow triangle points to a superclass. A filled diamond identifies exclusive lifecycle ownership, used for the collector's optional recorder. Plain directed associations represent stored references or nested values.
- Target-end multiplicities express values held by one source instance. A reverse `0..*` allows independent/shared DTO instances: nesting in JSON does not establish exclusive Python object ownership. Inheritance and module dependencies have no cardinalities.
- Rectangles marked module/function/hook are not invented runtime classes. External framework classes and repeated reference nodes are labelled explicitly. Only selected fields and operations appear; inherited Pydantic validation and serialization need not be repeated in every box.
- Every shown synchronous call opens a receiver activation and has a matching dashed reply ending it. Nested calls to the same participant open nested boxes. Callers stay active while waiting. Actor boxes denote participation, not CPU execution. Callback delivery is distinguished from GUI execution.
- `alt`, `opt` and `loop` denote alternatives, conditional interaction and repetition. Notes summarize omitted widget, validation or low-level I/O steps; they are not messages without activations.
- Activity diamonds have exclusive guards. State labels model logical workflow or derived fields, not an imaginary implemented state enumeration. Completing an entry activity is not shutting down the application.

The automated checker verifies class coverage and the explicit synchronous call/reply activation convention used here. Local rendering and review are still necessary; this checker is not a general-purpose UML theorem prover.

2 Architecture and Deployment

2.1 Component dependencies

Figure 1 separates browser presentation, backend services, compiler libraries, desktop presentation and local artifacts. Read dependency arrows from the consumer toward the used responsibility. React request helpers own cancellation and safe error presentation. Routes delegate validated mutations, retrieval, imports and coursework work rather than treating the provider as the compiler.

Both Python entry points call the same dataset module. The CSV symbol is a file artifact, not a relational database. Coursework uses the compiler and saved evidence; only explicit tutoring uses Gemini. Imports can produce candidates but cannot implicitly append a row. Browser speech

has its own provider boundary, independent of Gemini and desktop recording. The sixth browser workspace, Dictionary, reads versioned Markdown through `backend/dictionary.py`. These resources are not CSV corpus rows. Retrieval can use either or both local sources; collection statistics and compiler research continue to read only the saved CSV.

Requirements: FR-01, FR-06, FR-15–FR-16, FR-21–FR-36.

Source: `backend/main.py`, `backend/coursework_api.py`, `backend/import_api.py`, `frontend/src`, `data_collector/App.py`.

2.2 Deployment topology

Figure 2 distinguishes the trusted workstation, execution environments, artifacts and external services. Development uses browser–Vite HTTP and the Vite API proxy. The alternative built mode serves the versioned frontend assets and API directly through FastAPI. The desktop process can run independently, using the same local CSV and audio directory.

Keys remain in server configuration, not browser assets. Optional HTTPS provider requests and browser-selected speech services cross the local boundary. There is no authentication or authorization: bind the backend to loopback or an explicitly trusted boundary. CORS is not access control. If someone moves the backend elsewhere, its stored files are on that server, not the browser’s device. No cloud hosting service is silently assumed.

Requirements: NFR-04–NFR-06, NFR-08, NFR-13; FR-35.

Source: `backend/main.py`, `backend/config.py`, `frontend/vite.config.ts`, `frontend/src/voice.ts`.

3 Class Structure and Relationships

3.1 Desktop classes, optional devices and explicit errors

Figure 3 includes `App`, `Tooltip`, `Recorder`, `AudioError` and `EntryNotFoundError`. `App` inherits `CTk` and owns at most one recorder; missing audio imports do not prevent text collection. A recorder retains at most one stream, including a handle whose close must be retried. `start`, `stop` and `close` define its lifecycle.

Tooltips retain a widget and an optional popup. Bound callbacks retain these helpers; there is no invented `App.tooltips` collection. Dataset/audio functions are module dependencies. Error subclasses connect device/storage failures to truthful UI handling. Draft-owned filenames and captured arrays are distinct from committed CSV audio references.

Requirements: FR-08–FR-14, FR-17–FR-19; NFR-05, NFR-11.

Source: `data_collector/App.py`, `data_collector/audio_utils.py`, `data_collector/datasets.py`.

3.2 Compiler objects versus function-based algorithms

Figure 4 shows the actual namedtuple `Token`, `GrammarError` and private `_Transformations` class. The token class is generated, but is a real runtime class. Its two values represent the original token and assigned category. The transformation accumulator bounds work and retained before/after snapshots.

Grammar, productions, parse traces and most analysis results are dictionaries, lists or aliases. Tokenization, FIRST/FOLLOW computation and predictive parsing are functions; no fictitious `Grammar`, `Parser` or `LexerService` class is added merely to make a diagram look object-oriented.

Requirements: FR-15–FR-16, FR-29–FR-31.

Source: `compiler/lexer/tokenizer.py`, `compiler/parser/grammar.py`, `compiler/parser/analysis.py`, `compiler/parser/predictive.py`.

3.3 API request inheritance

Figure 5 makes the request hierarchy explicit. `RequestModel` rejects extra fields. Its bounded-text subtype is `AnalyzeRequest`; `TranslationRequest` extends that subtype with direction and options validation. Chat history associates a request with zero through twelve messages. The additional alternating-pair invariant restricts valid lengths to 0, 2, ..., 12 and at most 24,000 history characters. Constrained text aliases are not separate classes.

Requirements: FR-21, FR-24; NFR-14.

Source: `backend/schemas.py`.

3.4 Public analysis and translation results

Figure 6 connects a translation response to one analysis result, one coverage value and its evidence list. Analysis contains zero or more token results; translation content contains at most twelve vocabulary items. The response inherits the translation content rather than duplicating its fields. Chat also references evidence.

Evidence deliberately omits contributor, location, notes, timestamp and audio. Retrieval emits at most eight records, while the result DTO's list alone does not enforce that service-level bound. Associations are not composition: Pydantic objects can be reused by Python callers. Coverage indicates overlap, not an accuracy score.

Requirements: FR-23–FR-24, FR-29; NFR-08.

Source: `backend/schemas.py`, `backend/grounding.py`.

3.5 Collection DTOs and mutation errors

Figure 7 groups the collection request and response models. Create and patch requests differ from stored-entry, list, health and metadata responses. The `CollectionError` class carries safe failure details. A dataset response holds any number of entry DTOs. Aggregate counts describe the whole corpus, not only a search result. The same view also shows `DictionaryEntry` and `DictionaryResponse`. The latter associates zero through 100 entry values per response, enforced by Pydantic. It reports total source rows, matched rows, offset, limit and source filenames. The separate dictionary module reads references and raises the existing safe error type on failure; it does not use collection mutation methods.

PATCH requires at least one field, rejects explicit null, and preserves omitted fields. Only editable values and the review decision are sent by the web editor. The API DTO is not the persistence representation: CSV functions use dictionaries. Health reports configured capability, not a successful live Gemini probe. Error objects carry safe status/detail values.

Requirements: FR-21–FR-22, FR-25.

Source: `backend/schemas.py`, `backend/collection.py`.

3.6 Import candidates and coursework models

Figure 8 groups four import models and five coursework models. A preview can carry at most 100 structured candidates; an AI suggestion result at most twelve. Candidates remain unreviewed. Importing an approval flag cannot bypass the review boundary.

The profile has exactly three member strings; blanks are permitted for an unfinished group, while nonblank names must be distinct. Grammar requests extend to parse requests, then tutoring requests. Screenshot requests contain names and bounded data URLs; decoded image limits are enforced by storage logic, not inferred from string length alone.

Requirements: FR-26–FR-28, FR-32, FR-36.

Source: `backend/import_models.py`, `backend/coursework_models.py`.

3.7 Provider wire-response classes

Figure 9 follows the provider envelope from `GenerateResponse` to candidates, optional feedback, optional content and content parts. Optional values justify 0..1; lists justify 0..*. These classes validate upstream data, not corpus entities. The service requires acceptable finish status and nonempty non-thought text; a transport-level success alone is not a usable application answer.

Requirements: FR-24, FR-27–FR-28, FR-36; NFR-14.

Source: `backend/gemini.py`.

3.8 Provider service, settings and local grounding

Figure 10 shows the real `GeminiService`, `Settings`, `AIError` and `Grounding` dataclass. Each service references one settings object and one injected HTTP client. The application lifespan owns client creation/closure, so that link is not service composition. Grounding references evidence and coverage and stores exact-match/ambiguity results.

Retrieval remains a module of deterministic functions. It is neither training nor a hidden translation model. Safe provider errors remain distinct from successful translations and explanations.

Requirements: FR-23–FR-24; NFR-08, NFR-14.

Source: `backend/config.py`, `backend/gemini.py`, `backend/grounding.py`.

3.9 Frontend runtime class and functional modules

Figure 11 identifies `ApiError` as the only authored TypeScript runtime class. Its status accompanies a normal JavaScript error. React components, hooks and interfaces are not misrepresented as classes.

Request hooks own cancellation, pending/error state and protection against late results. App coordinates hash navigation and draft handoff. Shared dialogs open before focusing the requested field, wrap Tab navigation, prevent dismissal while busy and restore an existing trigger on unmount. The desktop sidebar scrolls on short screens. Voice remains opt-in.

Requirements: FR-35; NFR-15.

Source: `frontend/src/api.ts`, `frontend/src/useRequest.ts`, `frontend/src/components.tsx`, `frontend/src/App.tsx`.

4 Data Design, Transactions and Review Rules

4.1 Storage schema and compatibility

Canonical CSV order is `id`, `text`, `entry_type`, `french_gloss`, `english_gloss`, `category`, `source_location`, `notes`, `audio_filename`, `contributor`, `timestamp`, `language`, `review_status`, `lexical_category`. The supported eleven-field prefix lacks review metadata; the ten-field legacy format also lacks source location. Exact header order and row shape are checked. Reads normalize legacy metadata in memory; a later mutation writes the fourteen-field schema. Stored wording is not silently normalized for matching.

Language is Francanglais, Pidgin, mixed or unspecified. Approval is a local review decision, not a verified identity or certificate. Eligible reviewed Word annotations can affect lexical classification; mixed/unspecified or conflicting labels are not silently promoted to trusted evidence. Desktop and web forms choose unspecified/unreviewed for new drafts; the API's standalone create DTO retains its documented Francanglais default when language is omitted.

4.2 Transactional and identity boundaries

A path-keyed reentrant `FileLock`, with ten-second timeout, protects complete cooperative transactions. Every mutation writes a sibling temporary file, flushes/fsyncs it and uses `os.replace`. It is not a direct append. Pre-replacement failures preserve the old CSV. External manual writers and semantic last-writer-wins editing are outside that guarantee.

Desktop IDs are ten UUID hexadecimal characters; API IDs are full UUID strings. Desktop timestamps are local ISO seconds; API timestamps carry UTC offset. IDs/timestamps survive editing. The file is not a database enforcing every identity invariant: manually duplicated IDs can break GUI row identifiers and must not be treated as supported concurrent editing.

4.3 Reference vocabulary is not a research corpus

The supplied `dictionary/camfranglais.md` and `dictionary/extra_lexicon.md` contain 143 and 36 source rows, respectively. Each row retains its headword, original English meaning, topic, indicative etymological origin, source filename and line number. All 179 rows are retained, including repeated headwords, identical duplicates and distinct senses. This count is not a unique-word or collected-statement count.

The loader validates the three-column Markdown tables. Search normalizes matching text only; displayed wording is unchanged. Explicit slash alternatives, optional parenthesized wording such as `Na wa (oh)`, and terminal question/exclamation-mark variants support lookup. IDs include the filename and row line, not corpus UUIDs; editing source layout may change these IDs. The two files are loaded afresh on requests, so a corrected source can be retrieved without a stale-cache fallback.

The sources contain English meanings, not French translations or trusted part-of-speech annotations. Etymological “French”/“Pidgin” labels do not change the declared Francanglais reference language. No dictionary operation inserts CSV rows, approves records, adjusts learner labels or inflates coursework statistics. Export includes these files as implementation resources, separate from corpus artifacts. The desktop collector remains CSV-based.

Requirements: FR-37; NFR-08, NFR-16.

Source: `backend/dictionary.py`, `frontend/src/Dictionary.tsx`, `backend/coursework_export.py`.

4.4 Audio and review ownership

An entry references zero or one filename; a filename is not a foreign key or exclusive object owner. Only files created for the current unsaved draft enter its ownership set. Replacement/Clear/close remove those files with explicit cleanup errors. A committed name is removed from draft ownership before reset. Deleting a saved record retains its recording for deliberate separate cleanup.

The recorder distinguishes active capture, unfinished stopping and captured samples awaiting publication. Save/attach/play guards prevent silent loss. An approval snapshot must equal the current evidence values, including queued GUI edits; editing evidence invalidates approval unless the exact revised values are explicitly reviewed. These protections do not authenticate reviewers.

5 Use Cases, Activity and State Models

5.1 User goals and optional extensions

Figure 12 groups contributor/maintainer and learner/reviewer goals inside the local platform. Solid actor associations indicate participation, not permissions. Optional local lookup and optional AI suggestion extend translation under explicit guards; cloud transcription extends preview only when needed and consented. The dashed extension arrow points toward its base use case. No include/extend relation turns review into automatic approval. Export is a draft-generation goal, not a completed research submission.

Requirements: FR-02–FR-18, FR-23–FR-36; NFR-13, NFR-16.

Source: UI commands and API routes.

5.2 One collection/save activity

Figure 13 reads from start to final node. Unfinished capture and blank text terminate the attempt with the draft preserved. Successful validation enters the locked mutation. A failed commit retains the draft/audio; a successful one releases committed ownership and resets the form. Cleanup and refresh failures are reported separately, never described as rollback. The final node ends this attempt only. Desktop duplicates remain advisory.

Requirements: FR-02, FR-06–FR-10, FR-17–FR-19.

Source: `App.save_entry`.

5.3 Logical entry and selected-editor states

Figure 14 follows an unsaved draft through creation, selection, edited values and confirmed deletion. Choice guards are exclusive. Stale-ID/write failures return to a recoverable editor, not a fictitious saved state. A selected editor may outlive a row removed by another process. Successful deletion clears selection but leaves saved audio. These are conceptual row/editor states; there is no corresponding enum in source.

Requirements: FR-02, FR-12, FR-18–FR-19, FR-22.

Source: desktop save/edit/delete handlers.

5.4 Audio readiness and retry states

Figure 15 isolates the device lifecycle from row persistence. Ready may already have an attachment. Failed start cleanup or failed stop enters a retry state retaining usable handles/frames. Completed capture attempts WAV publication; write failure retains the array for retry without reopening the microphone. Clear is blocked during active recording, while confirmed close attempts resource cleanup. A failed close keeps the window available. A published WAV is still only staged until an entry is saved.

Requirements: FR-08–FR-10, FR-17–FR-19; NFR-05.

Source: Recorder, desktop audio handlers.

6 Sequence Views: Detailed Interaction Explanations

6.1 Attachment and collection commit

Figure 16 separates optional attachment from entry creation. Copy failure preserves the previous attachment; success stages only a generated name. Save first checks unfinished audio and trimmed text, then passes a reviewed field snapshot to the dataset transaction. Only commit success releases the audio reference from draft ownership and resets the form. One subsequent snapshot drives all collection views. Refresh/cleanup errors do not retroactively turn a committed save into a failed write.

Requirements: FR-02, FR-06, FR-09–FR-10, FR-17, FR-19.

Source: `App.attach_audio_file`, `App.save_entry`, `dataset.append_entry`.

6.2 Search and cached selection

Figure 17 loads before replacing the visible tree. A read error preserves the cache and editable draft. On success, case-folded substring matching covers all stored values; the ID cache retains all rows, including those hidden by the current filter. Selection uses this snapshot without a second CSV read. A disappeared selected ID produces feedback while preserving unsaved edits. An empty result is not a disguised storage failure.

Requirements: FR-11, FR-18; NFR-10.

Source: `App._refresh_browse_list`, `App._on_browse_select`.

6.3 Changed-field editing

Figure 18 rejects absent selection/blank text, computes changes against the loaded values and applies the exact-snapshot review rule. The dataset layer finds the ID while locked and rejects stale updates without rewriting. Expected failures preserve editor values. A successful update synchronizes the local snapshot and refreshes views, reporting later refresh failure independently. Unchanged identity, contributor, timestamp, whitespace and audio metadata are preserved by the desktop editor.

Requirements: FR-06, FR-12, FR-18–FR-19, FR-22.

Source: `App._save_browse_edit`, `dataset.update_entry`.

6.4 Confirmed deletion

Figure 19 places the confirmation before any mutation. No/dismissed leaves the row, selection and media untouched. Yes invokes the locked deletion; missing IDs and storage failures cannot produce a success message. Only a committed deletion clears selection and refreshes views. The old saved recording is retained. Atomic replacement is not an undo system, and confirmation is not authorization.

Requirements: FR-12, FR-18–FR-19.

Source: `App._delete_browse_entry`, `dataset.delete_entry`.

6.5 Recording, callback delivery and publication

Figure 20 first stops in-process playback, then opens mono 44,100 Hz capture. Failed startup restores a truthful inactive state; failed closure retains its handle for retry. Callback calls execute on the device thread between GUI events and copy frames only while recording. Stop returns samples, no samples or an explicit error. No samples preserve any old attachment. WAV-write failure retains captured samples; retrying publication does not start another recording. Save remains blocked while capture work is unfinished. Physical audibility is a separate acceptance gate.

Requirements: FR-08, FR-10, FR-19; NFR-05.

Source: `Recorder.start`, `Recorder.stop`, `App._save_captured_audio`.

6.6 Playback and external fallback

Figure 21 requires a filename and finished capture. Missing files return failure. Available libraries read/play audio; an unavailable or failed library path may launch the OS default player. The return value means that a request was accepted, not that sound was heard to completion. The GUI reports rejection or expected audio errors. In-process playback can be stopped during cleanup; the lifetime of an externally launched player is not controlled. No CSV mutation occurs.

Requirements: FR-13; NFR-03.

Source: `audio_utils.play_audio`, `App._play_audio_filename`.

6.7 Statistics from one snapshot

Figure 22 replaces the old three-read design. A successful read supplies a total and distributions by type, topic, language and review; batch refresh passes the same rows to other views. Read failure leaves prior bars intact. Each group sorts counts descending, uses a safe denominator and handles an empty corpus explicitly. The recorded benchmark measures these actual active handlers plus Tk idle rendering, not an isolated count helper.

Requirements: FR-14, FR-17; NFR-10.

Source: `App._refresh_stats`, `App._refresh_views`.

6.8 Shared CSV transaction

Figure 23 expands the common persistence boundary. Lock acquisition can time out before mutation. Within a successful acquisition, the same reentrant lock supports nested reads, ID/duplicate checks and mutation. Only completely written/fsynced sibling files reach atomic replacement. The finally path removes an incomplete sibling and context-manager exit releases the lock. Missing update/delete IDs never become successful rewrites. Lock activations show acquire/release execution, not the whole ownership interval.

Requirements: FR-01, FR-06, FR-19, FR-25; NFR-11.

Source: `data_collector/dataset.py`, `backend/collection.py`.

6.9 Local-source translation

Figure 24 shows explicit submission, reviewed retrieval and three outcome paths. An unambiguous full-entry alignment returns local wording without tone synthesis. Otherwise enabled AI may receive only selected evidence; disabled AI produces actionable 422 guidance. Storage failure does not fall through to a cloud request. Successful text is analyzed locally and labelled with origin/model/evidence/coverage. Cancellation prevents stale display, not guaranteed rollback of provider work. Collection and dictionary use have independent switches, enabled by default. Reference aliases participate in full-entry matching; English definitions are not split into invented bilingual pairs. Distinct full-entry target meanings remain ambiguous, even across the collection/reference boundary. Identical targets can share source evidence. Missing dictionary French meanings cannot veto a valid approved French alignment. Local results distinguish `dataset`, `dictionary` and `local_sources`; AI results retain their AI designation, including `ai_with_sources`.

Requirements: FR-23–FR-24, FR-29, FR-35.

Source: `backend/main.py`, `backend/grounding.py`.

6.10 Chat history and provider boundaries

Figure 25 uses only completed successful exchanges for bounded history. Optional retrieval uses whitelisted evidence. Missing keys fail before a network request; configured requests have a total deadline and at most one bounded retry for an initial 503. Safe errors distinguish quota, transport, timeout, blocked/truncated output and invalid content. Only a current successful result appends a user/assistant pair. Failed turns retain the draft. Clearing browser history does not erase provider-retained data. The provider service rechecks each evidence source against its own switch; turning collection use off must not strip enabled reference evidence, and turning dictionary use off must exclude it. Citation metadata is separate from private collection provenance. A reference is never described as approved fieldwork.

Requirements: FR-24, FR-35; NFR-08, NFR-14.

Source: `frontend/src/Assistant.tsx`, `backend/gemini.py`.

6.11 Read-only dictionary lookup and handoff

Figure 29 shows a debounced, cancellable search, validated route parameters, both file reads and strict parsing before a paginated response. The total and matched counts refer to dictionary rows, not corpus statistics. Missing files or malformed rows produce a visible 503; invalid query parameters produce 422. Neither failure silently becomes an empty dictionary.

Opening a listed form invokes the App handoff callback: the translator gets a draft with Francanglais-to-English direction and dictionary lookup enabled. The user must still press Translate. There is no implicit AI call or collection save. Receiver activations and matching replies show every synchronous communication, including nested parser calls and the App callback.

Requirements: FR-35, FR-37; NFR-15–NFR-16.

Source: `frontend/src/Dictionary.tsx`, `frontend/src/App.tsx`, `backend/main.py`, `backend/dictionary.py`.

6.12 Import preview, suggestions and explicit saving

Figure 26 separates three actions. Preview validates a bounded upload and extracts supported documents locally. Media/OCR requiring cloud work must have explicit consent. A second action can request candidate suggestions; validated candidates remain unreviewed exact source quotations. Only an existing candidate deliberately opened, reviewed and saved reaches collection mutation. Translate/assistant handoff fills a draft without submitting. Original imports are not retained as managed collection files. A failed preview does not authorize an automatic candidate save.

Requirements: FR-26–FR-28.

Source: `backend/import_api.py`, `backend/imports.py`, `frontend/src/Imports.tsx`.

6.13 Grammar analysis and predictive parsing

Figure 27 groups function-module interactions. Corpus analysis obtains a bounded saved snapshot before computation; practice parsing supplies one entered text. Grammar validation/transformation precedes FIRST/FOLLOW, table/conflict computation and category-token parsing. Lexical/storage failures prevent parsing. Conflicts or residual recursion produce rejection instead of choosing an arbitrary production. The stack machine records pre-action snapshots and requires complete input/end-marker agreement. Analysis uses current editor grammar without saving it. No provider decides acceptance, and acceptance is not proof of authentic speech.

Requirements: FR-29–FR-33; NFR-06, NFR-14.

Source: `backend/coursework.py`, `backend/coursework_api.py`, `compiler/parser`.

6.14 Saved coursework snapshot and export

Figure 28 begins with a profile snapshot sent to the API. Grammar validation and locked atomic JSON replacement precede the returned `ProjectProfile`. The frontend marks that submitted snapshot saved, not edits made while awaiting the response.

Export is a separate GET disabled during unsaved/busy work. It reads saved profile/grammar, current CSV and stored screenshots, computes artifacts and assembles a no-store ZIP response. Browser code validates a nonempty ZIP Blob, starts a named download and releases its object URL. No unsaved practice input or AI answer is included. Individual files are protected, but export is not one global multi-file transaction. A 25-section draft is not a certified 25–30-page research report, and slides still require personalization/rehearsal.

Requirements: FR-32–FR-34; NFR-08, NFR-16.

Source: `backend/coursework_store.py`, `backend/coursework_export.py`, `frontend/src/CourseworkEvidence.tsx`.

7 HTTP Interfaces and Failure Contracts

7.1 Complete application route inventory

The table covers all twenty-one application method/path combinations exposed by the generated OpenAPI schema. FastAPI's schema/docs and built static assets are framework/presentation routes, not additional data-mutation APIs.

Method	Path	Successful contract
GET	/api/health	200 HealthResponse; configuration, no live probe.
GET	/api/metadata	200 MetadataResponse.
GET	/api/dataset	200 DatasetResponse; optional search, global counts.
GET	/api/dictionary	200 DictionaryResponse; query, offset and bounded limit; no mutation.
POST	/api/dataset	201 DatasetEntry; validated create.
PATCH	/api/dataset/{entry_id}	200 DatasetEntry; changed editable fields.
DELETE	/api/dataset/{entry_id}	204; no response body.
POST	/api/analyze	200 AnalysisResult; deterministic local lexer.
POST	/api/translate	200 TranslationResponse with origin/evidence.
POST	/api/chat	200 ChatResponse.
POST	/api/imports/preview	200 ImportPreview; multipart file/consent.
POST	/api/imports/suggest	200 SuggestionResponse; no save.
GET	/api/coursework	200 saved profile, brief, requirements, stats and screenshots.
PUT	/api/coursework/project	200 ProjectProfile; not a boolean-only acknowledgement.
POST	/api/coursework/analyze	200 grammar, lexical report, tests and readiness.
POST	/api/coursework/parse	200 tokens and explicit parse result.
POST	/api/coursework/explain	200 ChatResponse; optional unverified tutoring.
POST	/api/coursework/screenshots	201 generated image ID/name metadata.
GET	/api/coursework/screenshots/{image_id}	200 decoded/re-encoded PNG.
DELETE	/api/coursework/screenshots/{image_id}	204; no response body.
GET	/api/coursework/export	200 application/zip; attachment; Cache-Control: no-store.

7.2 Validation, errors and bounded work

JSON request models reject extra fields and invalid lengths/values. PATCH rejects null/empty updates. Relevant failures include 400 invalid import/consent, 404 missing entry/image, 409 duplicate conflict, 413 oversized input, 415 unsupported formats, 422 validation/grammar/coverage problems, 429 quota, 500 local storage failure, 502 invalid provider output, 503 busy/unconfigured service and 504 provider timeout. Not every route emits every code; safe detail messages remain actionable without returning secret-bearing provider bodies.

Imports are limited to 12 MiB, 40 PDF pages, 40,000 extracted characters and 4,000-character segments. Corpus analysis bounds 500 entries, 100,000 total text characters and 4,000 per entry. Grammar input is bounded to 12,000 characters with additional expansion/snapshot/parse guards. Screenshots are limited to six, 2 MiB and twelve megapixels each; accepted images are re-encoded.

The client also guards pending/listening/blank states but is not the only validation boundary.

An aborted or timed-out mutation may already have committed. The frontend preserves drafts and prompts refresh rather than promising rollback or blindly duplicating a request. A reported local approval or readiness flag is never an authenticated permission, independent research verification or grade.

8 Verification, Recovery and Human Acceptance

8.1 Recorded engineering checks

Evidence	Result and limit
Python regression run	268 tests passed, including 16 dictionary tests, 56 desktop tests and 33 maintenance/checker tests. Synthetic storage and mocked provider/device failures. Reference source files are read, never counted as fieldwork.
Frontend checks	86 component tests; 22 desktop/mobile Chromium workflows; app/test type-checking and production build passed. Browser API responses are controlled fixtures.
Real WAV round-trip	Silent mono 44,100 Hz WAV written/read with 4,410 frames. This exercises codec/file I/O, not a microphone or audible playback.
Desktop benchmark	1,000 synthetic 500-character rows, one warm-up and 20 measured runs per operation. Search p95 53.31 ms; Stats p95 606.01 ms, both below 1,000 ms on the recorded machine.
Recovery	Sixteen snapshot tests verify exact bytes, media/profile recovery, corruption rejection and refusal to overwrite an existing destination; five tests check timing/fixture methodology.
Documentation	Local UML/LaTeX generation; 49-class mapping, explicit activations, 29 source/image references, full-size sheet mapping, embedded pixels and reference/layout checks.

Reference integration evidence is recorded separately at `docs/evidence/dictionary-verification.json`. The two-worker frontend reruns passed without relaxed assertion timeouts after an initial concurrent run timed out; the dictionary browser selector was corrected to use the combobox's accessible role/name. No production timing workaround was added. The benchmark report at `docs/evidence/collector-benchmark.json` records source hashes, raw samples, runtime and approximately 31.8 GiB RAM. It does **not** certify the SRS's original 8 GB/local-SSD reference setup. An earlier intermittent Windows storage-test failure was not reproduced in 25 targeted retries; subsequent complete runs passed. No speculative retry patch or claim that its cause was proven is made.

8.2 Reproducible commands and CI boundary

From the repository root:

```

.\.venv\Scripts\python.exe -m tools.run_python_tests
cd frontend
npm test
npm run typecheck
npm run test:e2e
npm run build
cd ..

```

```
.\docs\build.ps1 -PlantUmlJar .\docs\tools\plantuml.jar  
.\venv\Scripts\python.exe -m tools.check_documentation
```

The root README gives dependency setup, isolated benchmark and backup commands. CI configures Windows Python 3.11/3.14, Node 22 frontend/browser checks and fresh locally rendered documentation. The workflow was linted locally; its existence is not evidence that a remote Actions run has completed.

8.3 Backup, verify and restore procedure

Close the collector/API and finish uploads/recordings before a snapshot. The snapshot CLI copies CSV bytes, managed audio including unreferenced files, saved profile and screenshots into a new destination with a SHA-256 manifest. Keys, locks and incomplete temporary files are excluded. Missing/corrupt media, unsafe paths, links and detected concurrent changes fail explicitly.

Verify before restoring, and restore only into another new directory. Inspect restored rows/media before deliberately copying verified data back with all applications stopped; copy CSV last. Never overwrite the live collection directly through the restore tool. Protect backups as personal research material. Checksums establish integrity, not authentic provenance or permission to share.

8.4 Remaining human and operational gates

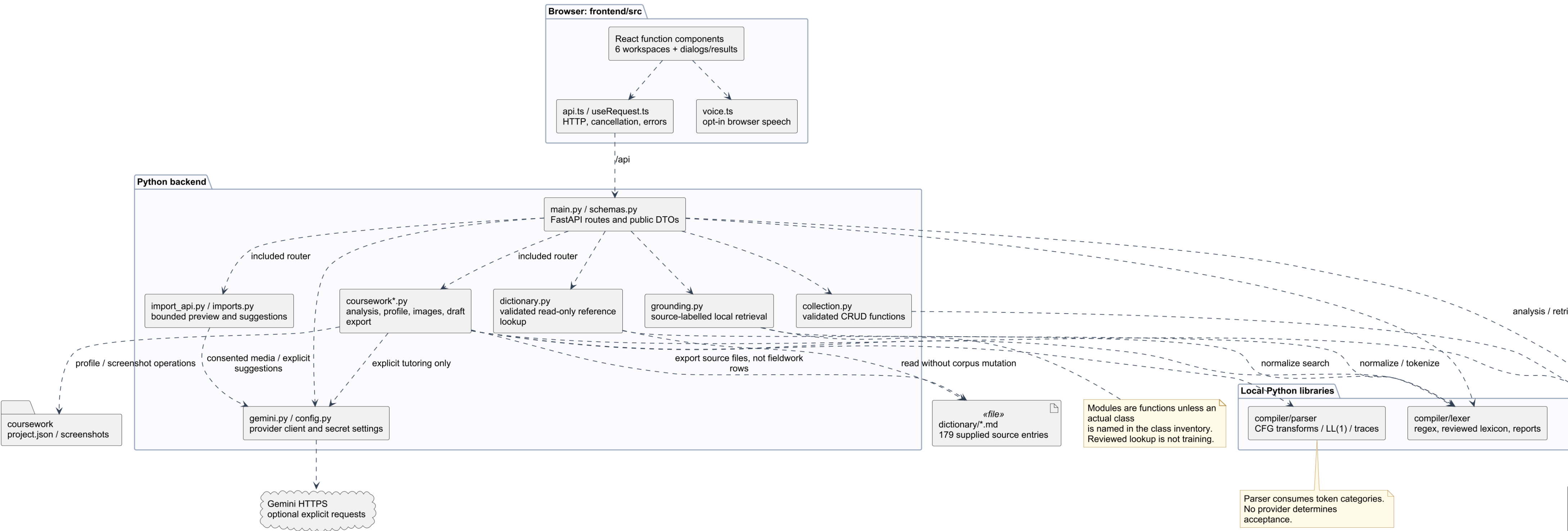
The group must supply genuine 10–15 manually transcribed statements across the ten topics, known meanings/provenance/permission, the other two names and matricules, and actual individual contributions. It must review classifications, justify grammar against that corpus, interpret outcomes, provide genuine screenshots and finish the separate 25–30-page research report and rehearsed presentation. No synthetic test fixture is fieldwork.

Physical recording/playback, real mobile/assistive-technology use, authorized live-provider trials, the original reference-machine benchmark and group acceptance remain explicit gates. A trained unseen-word predictor is deferred. These boundaries remain visible in the SRS product/sprint backlogs.

A Full-size UML Sheets

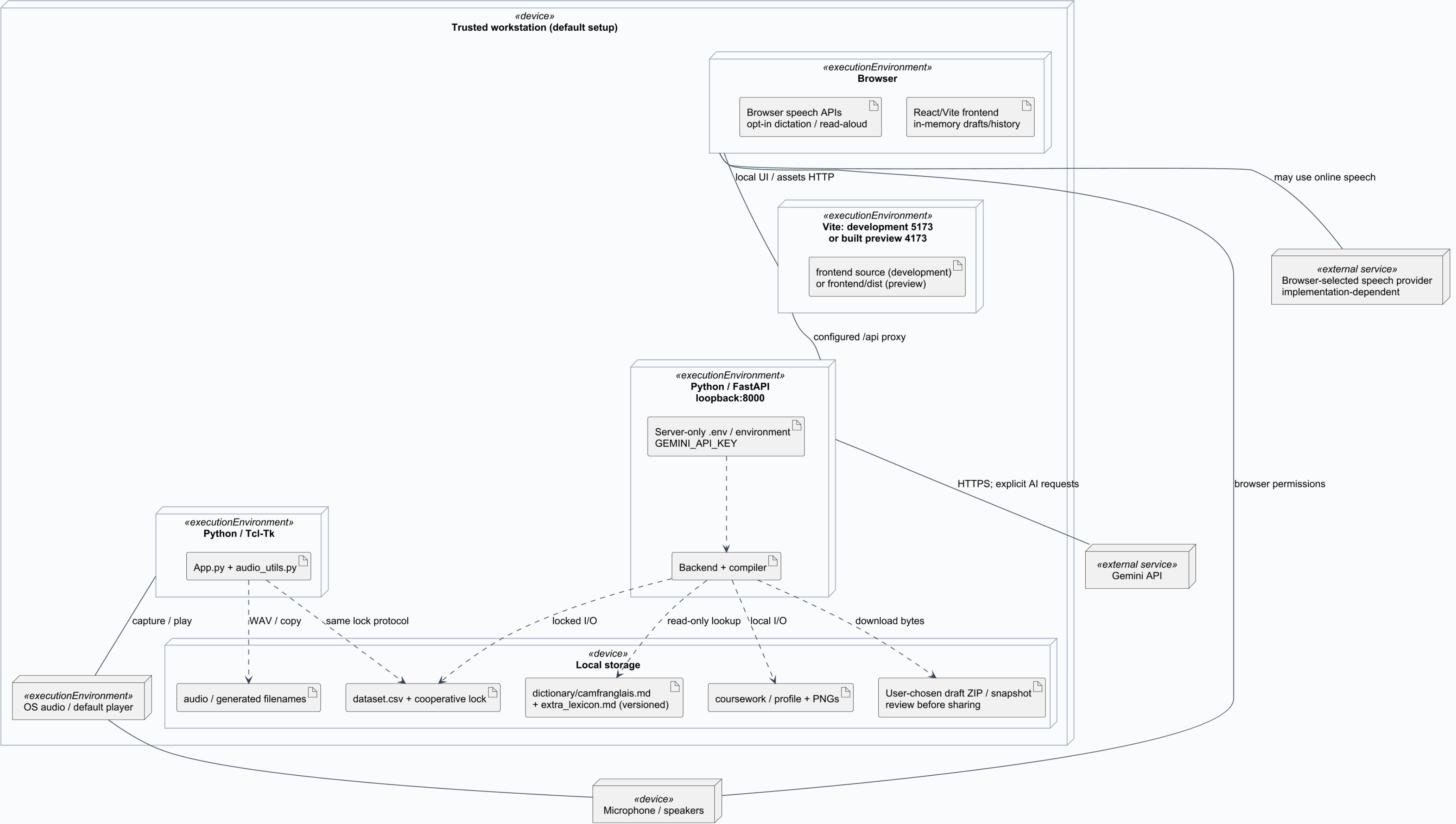
The following sheets are the rendered source views referenced throughout this document. Large paper boxes preserve approximately nine-point default diagram labels at actual size. A separate `uml-atlas.pdf` contains the same sheets without the explanatory text. Editable PlantUML and full resolution PNGs are retained in `docs/diagrams`.

Implemented package dependencies and data boundaries



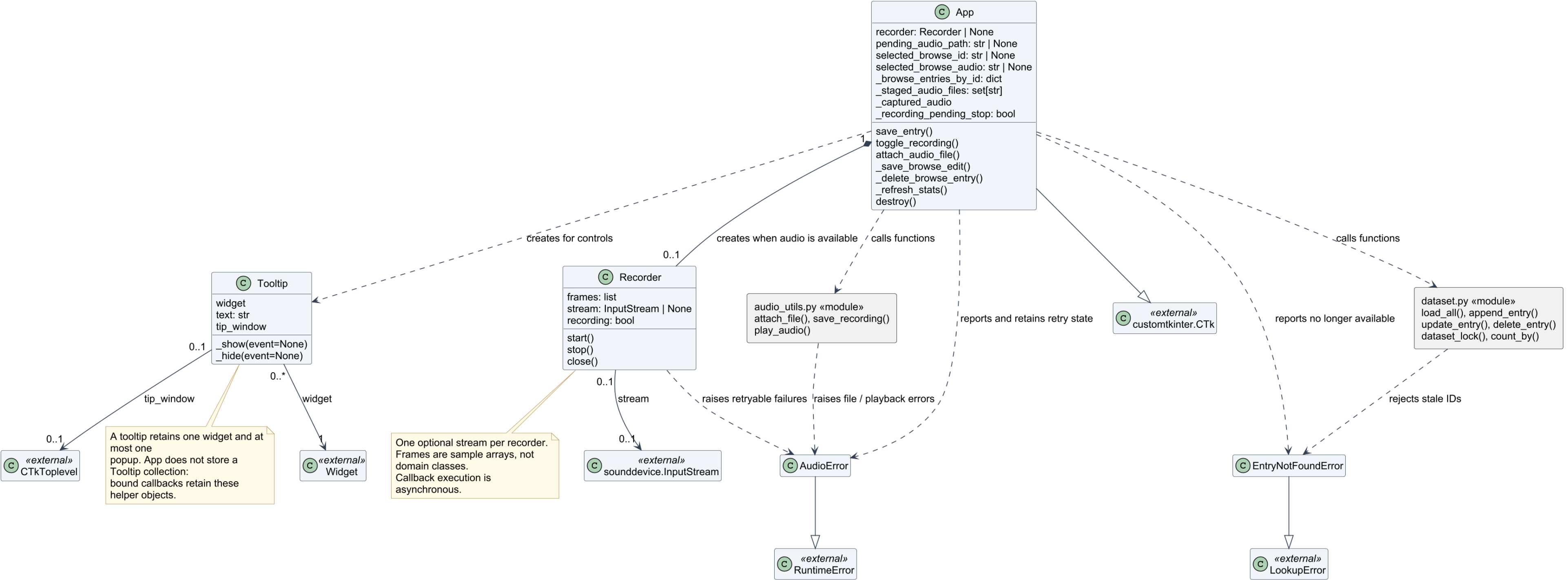
Dashed arrows are dependencies, not timed calls.
Browser speech may contact a separate browser provider (deployment view).
Imports do not automatically write CSV. Export reads saved evidence.

Trusted local deployment; explicit optional remote services



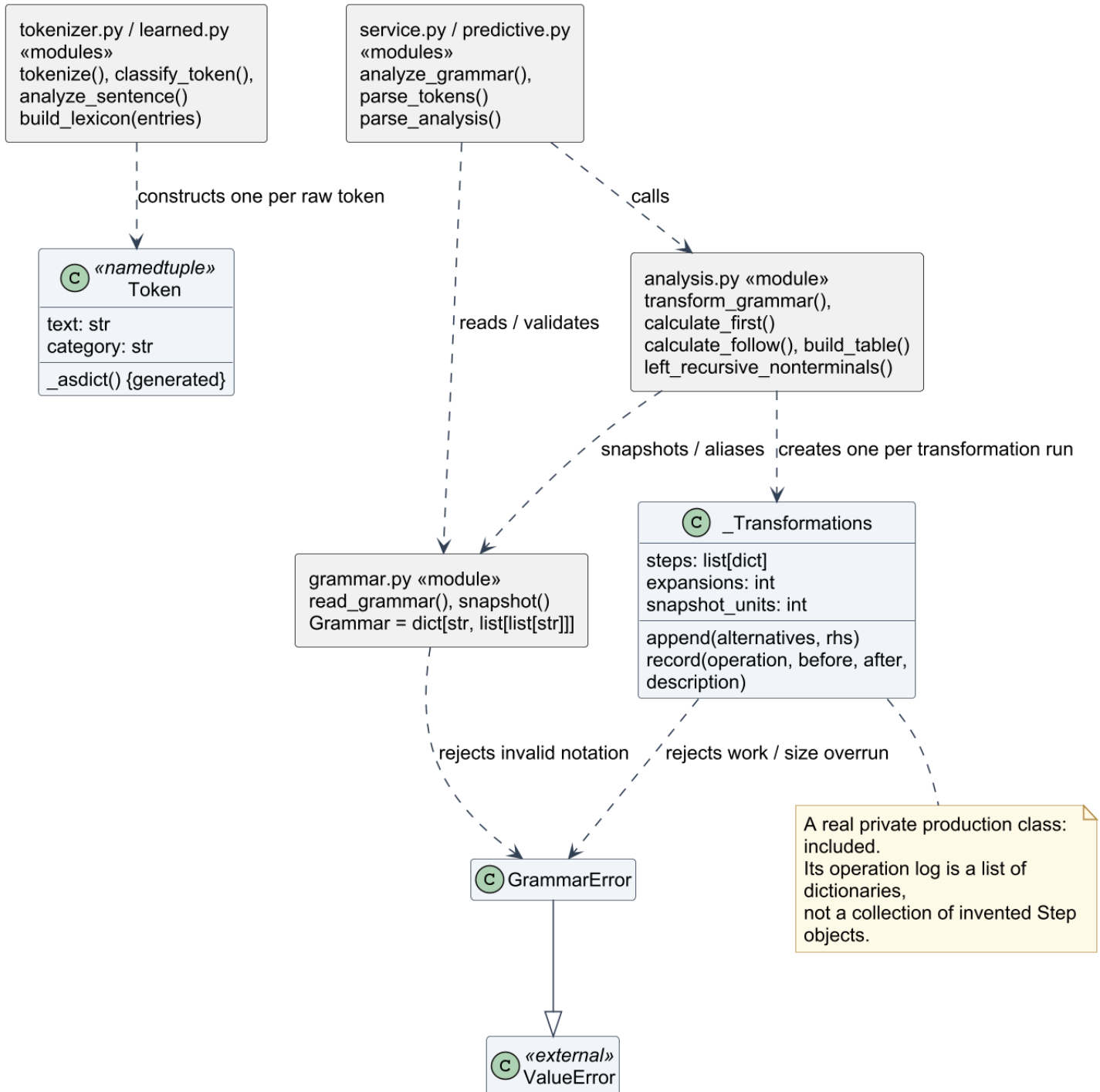
No authentication or public multi-user deployment is supplied.
CORS is not access control. Keep the backend on a trusted local boundary.
If the backend is remote, its CSV/audio/profile live on that server.
FastAPI exposes the API only; Vite serves development or built-preview UI.
Production hosting needs a separate static server and matching API routing.
AI/provider access is optional; local compiler, CRUD and exact reviewed lookup work offline.

Desktop application classes and audio ownership



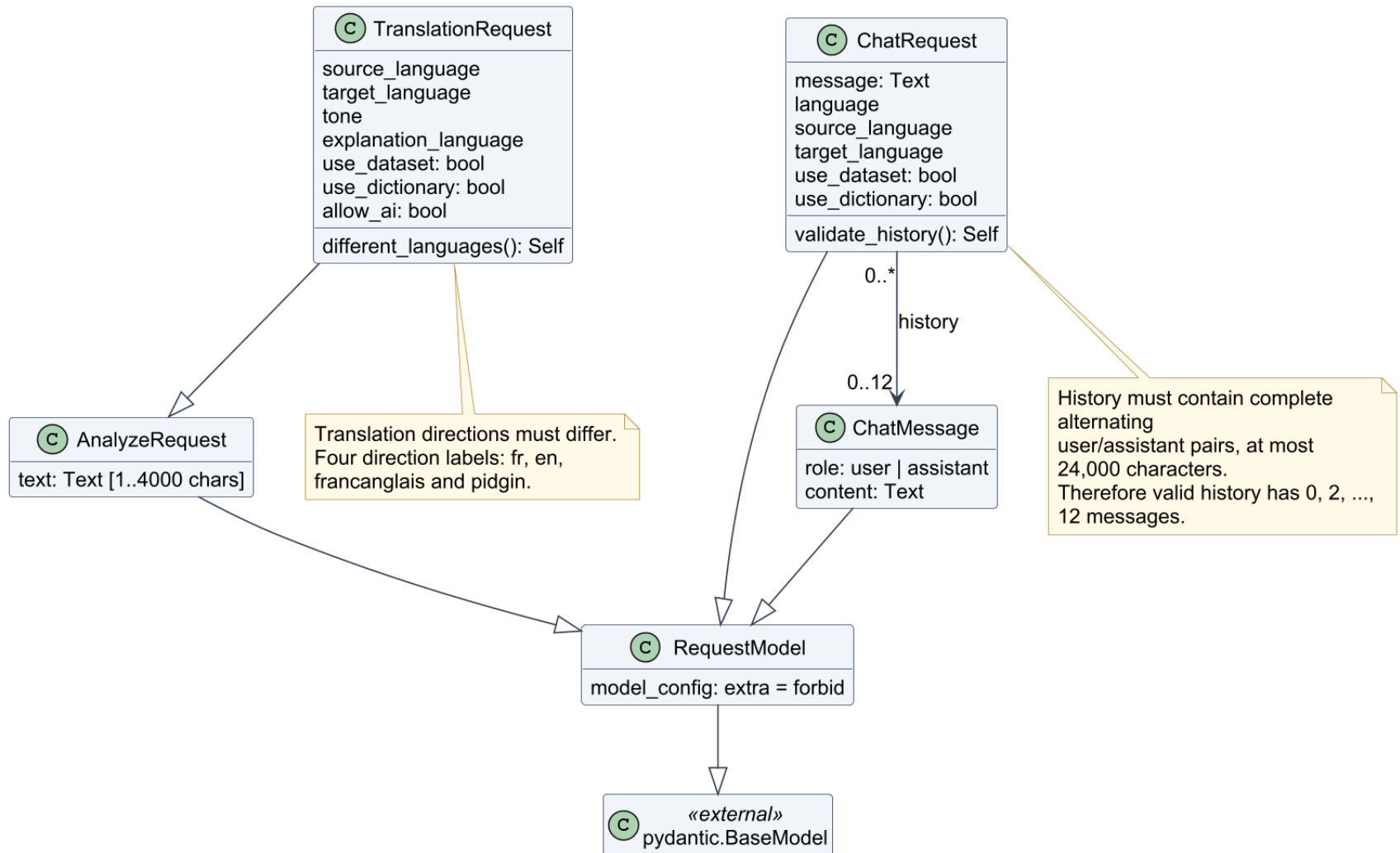
Five production classes; library types are explicitly external.
Per-instance target multiplicities express stored references.
Audio filenames are not object ownership or database foreign keys.

Compiler runtime classes and function modules



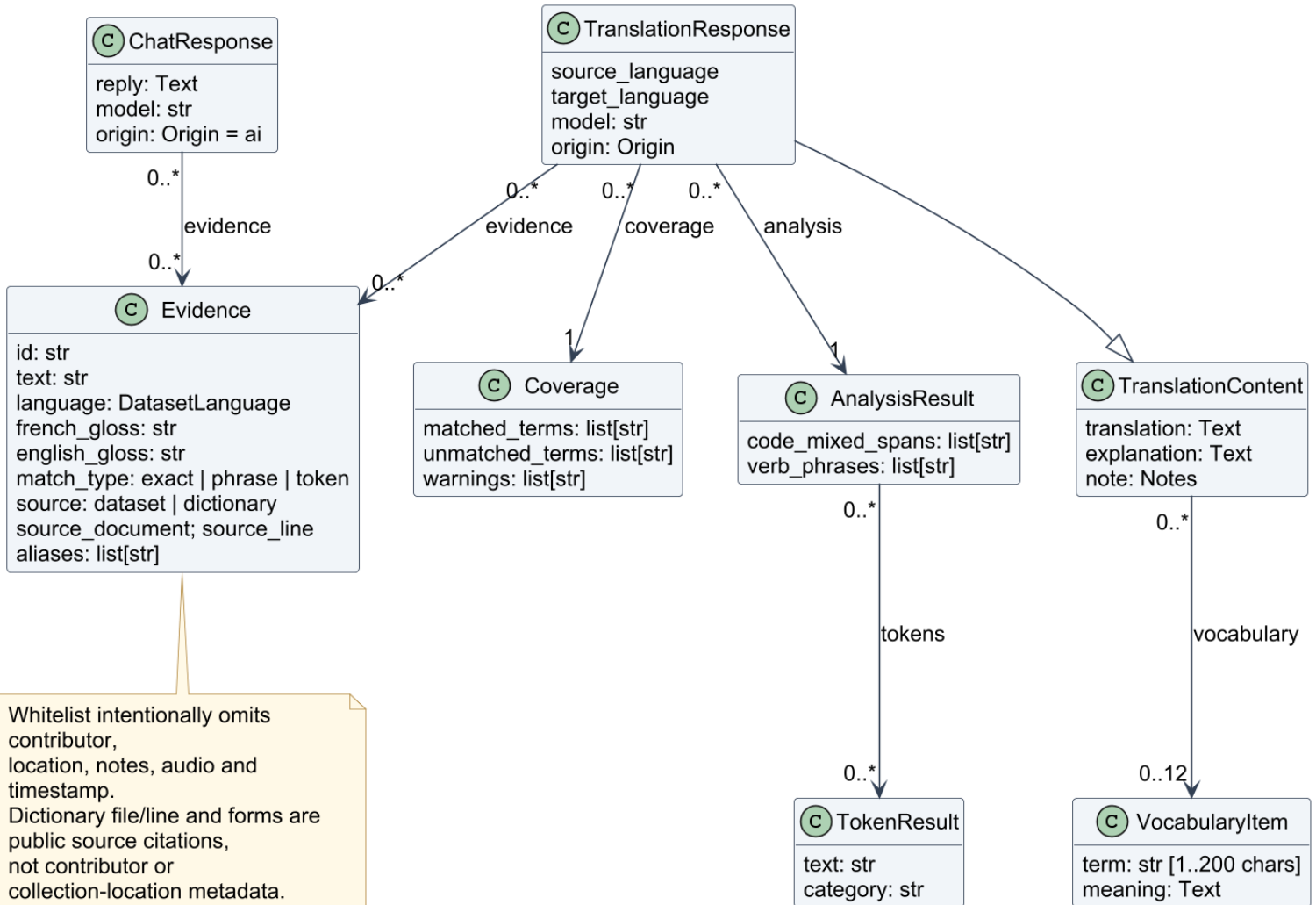
Token is an application-defined runtime class generated by namedtuple. Grammar, Production, analysis results and parse traces are aliases/dictionaries. There is no `LexerService`, `Grammar` class, `Parser` class or compiler database. Lifetimes of function-local values are not drawn as class compositions.

API request inheritance and conversation validation



All five primary classes live in backend.schemas.
Text is a constrained string alias, not an additional class.
Entry, import and coursework request subclasses are in detail views.

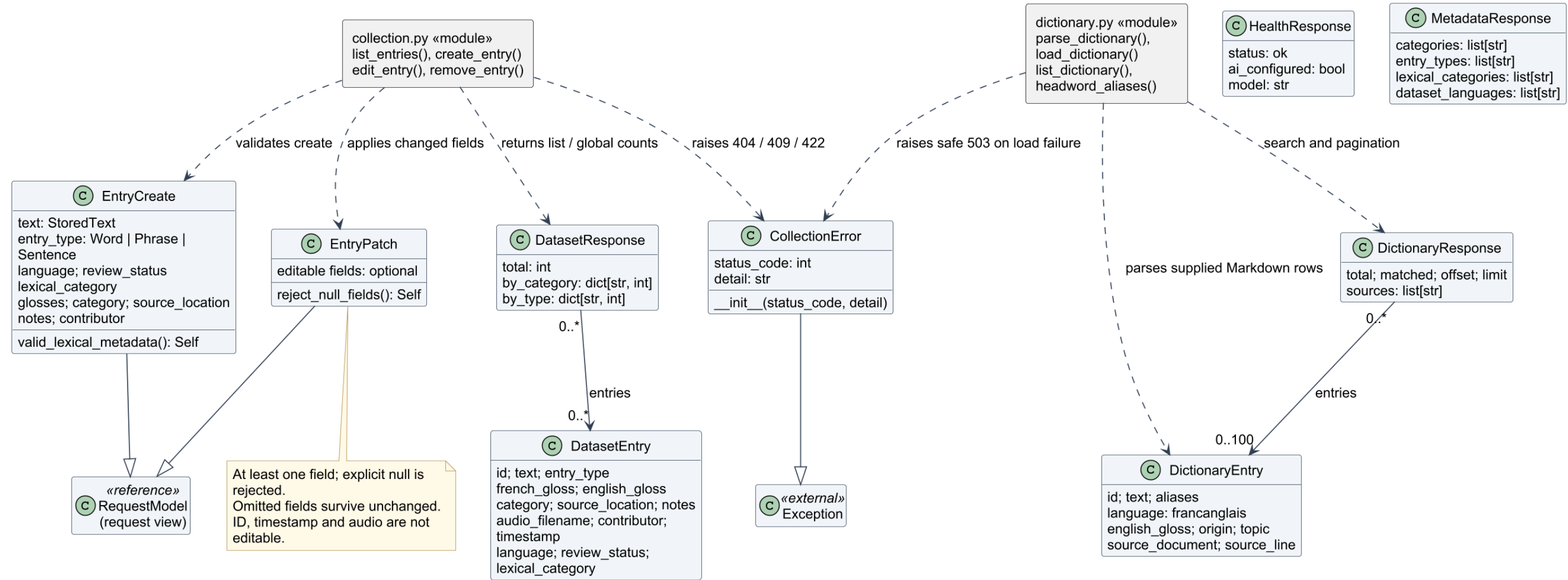
Translation and analysis result DTOs



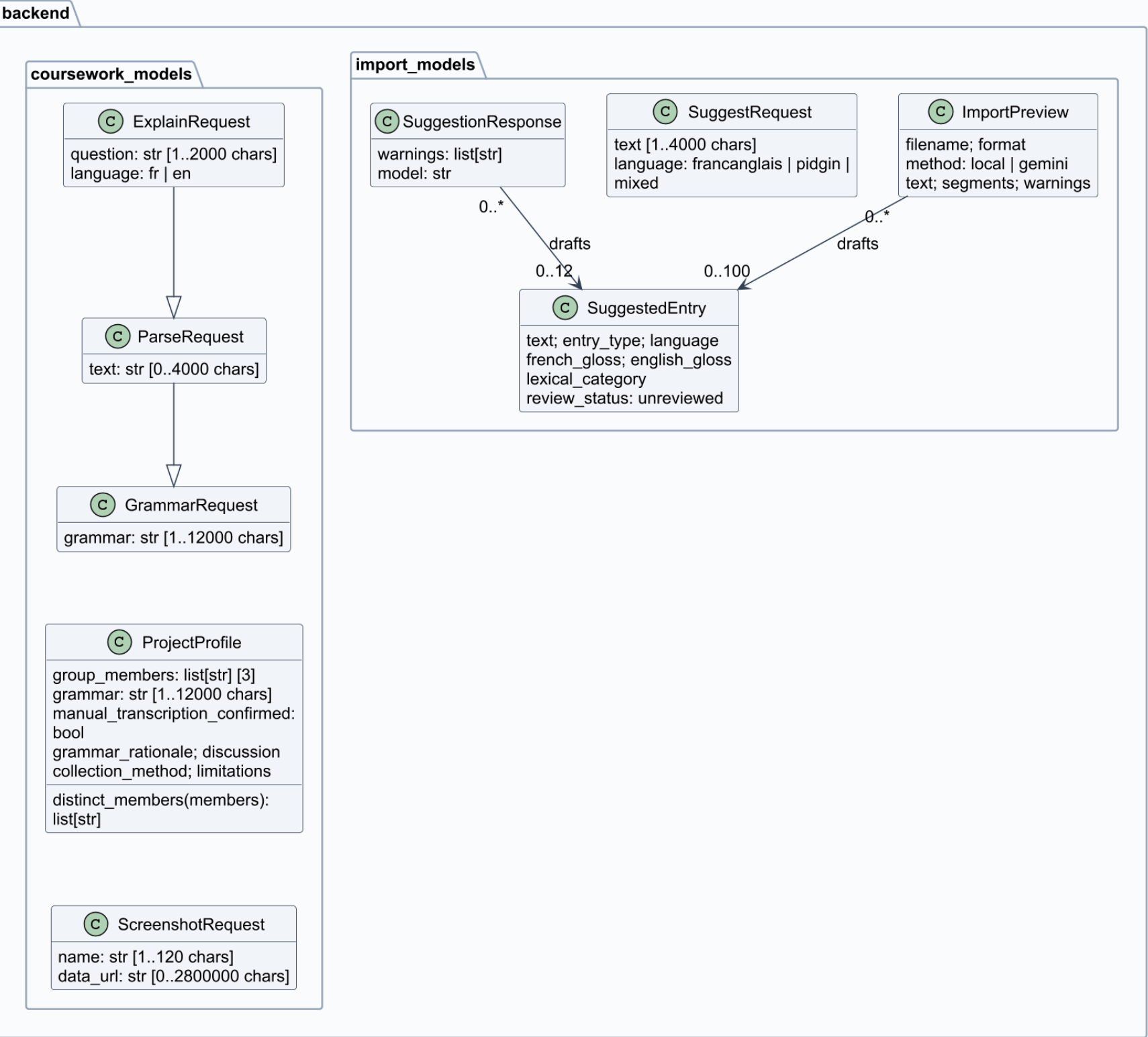
Whitelist intentionally omits contributor, location, notes, audio and timestamp. Dictionary file/line and forms are public source citations, not contributor or collection-location metadata. Retrieval emits at most 8 evidence records, but these DTO list types alone do not enforce 8.

All eight primary classes live in backend.schemas. Each directly extends pydantic.BaseModel except TranslationResponse. Associations describe nested values, not exclusive lifetimes; Pydantic values may be shared/reused by Python callers. Validation/serialization use inherited model_validate/model_dump.

Separate collection and reference dictionary DTOs

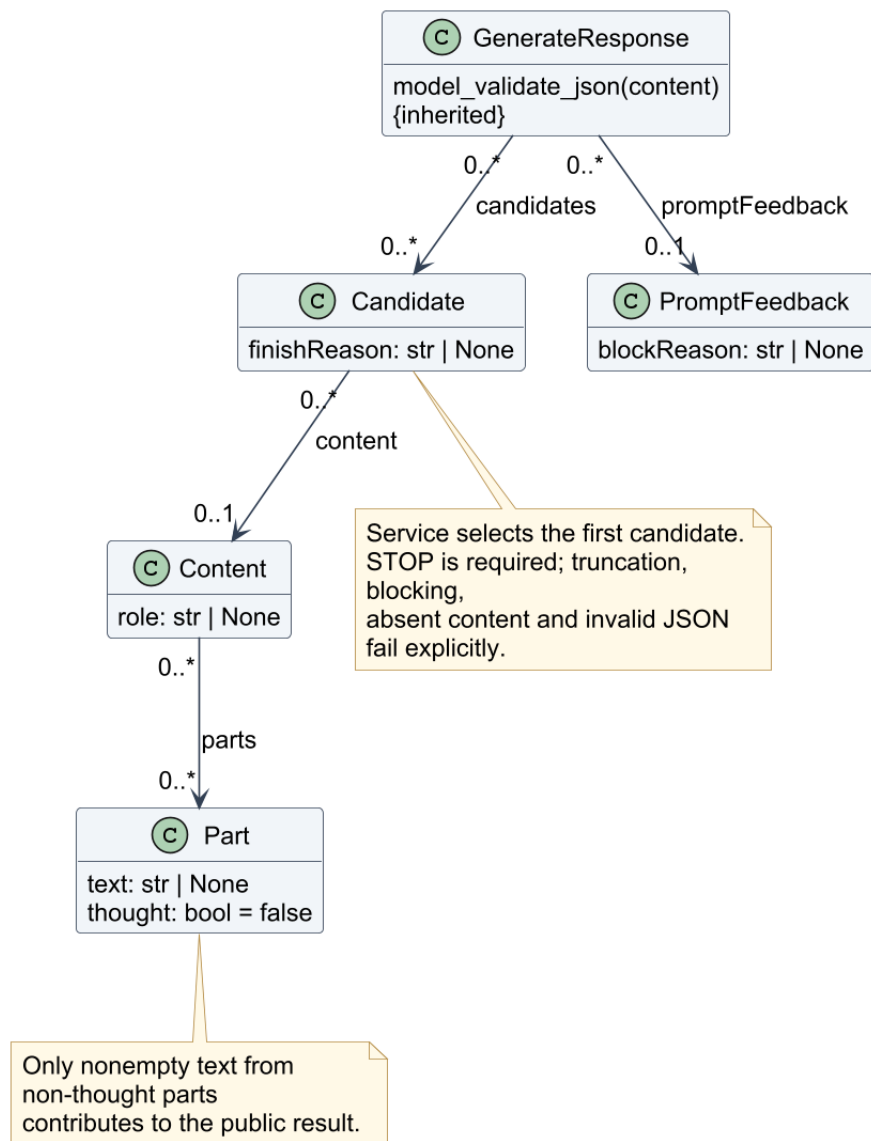


Import and coursework request/result classes



SuggestRequest and all coursework classes inherit RequestModel (directly, except ParseRequest and ExplainRequest as drawn). Other import classes directly inherit BaseModel. SuggestedEntry additionally forbids extra fields; approval cannot be imported. Three member strings may initially be blank; nonblank names must be distinct. UI type aliases/interfaces are not additional runtime model classes.

Gemini wire-response validation classes

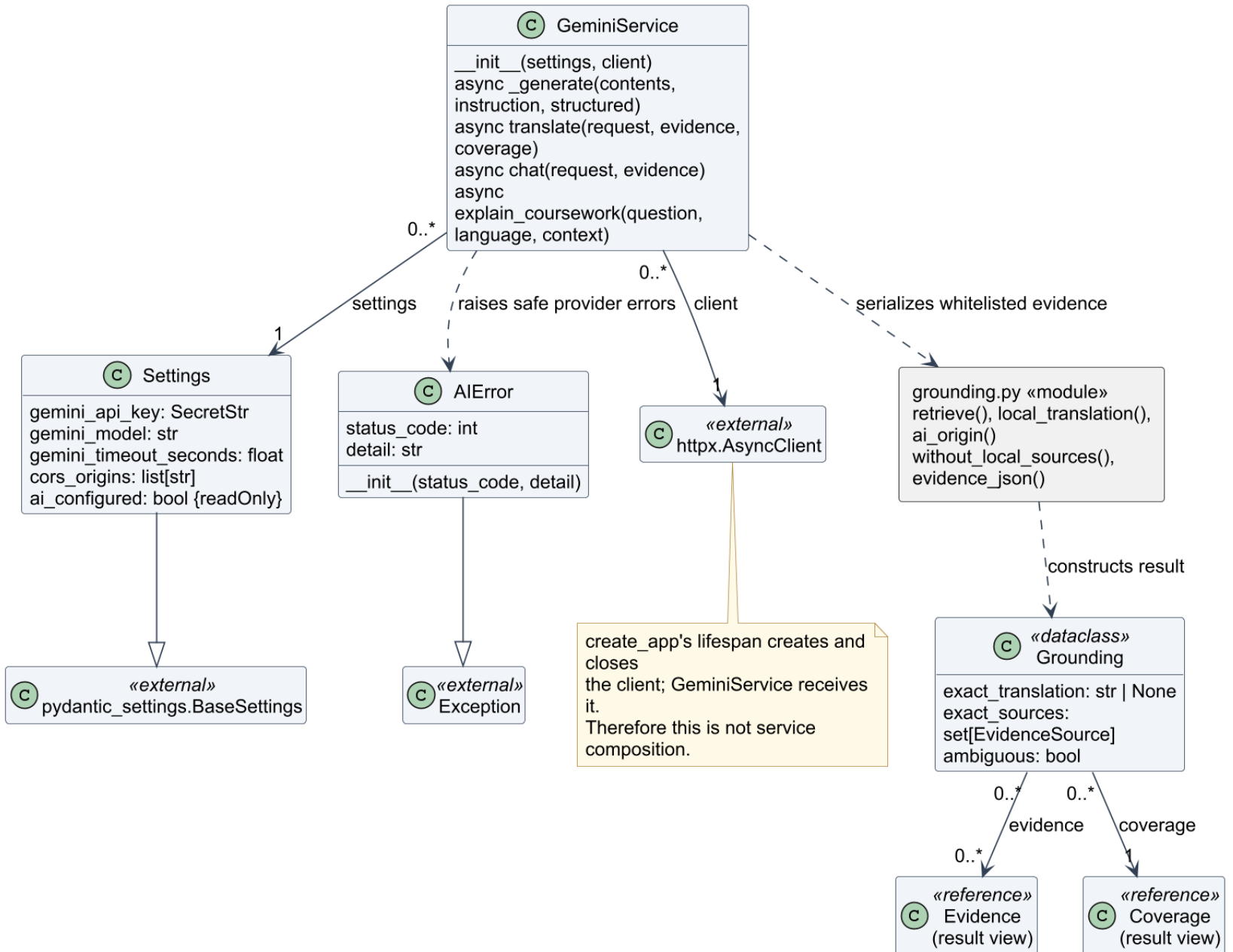


All five primary classes live in `backend.gemini` and extend `BaseModel`. Optional fields/list defaults justify 0..1 and 0..*.

They validate upstream wire data, not stored corpus entities.

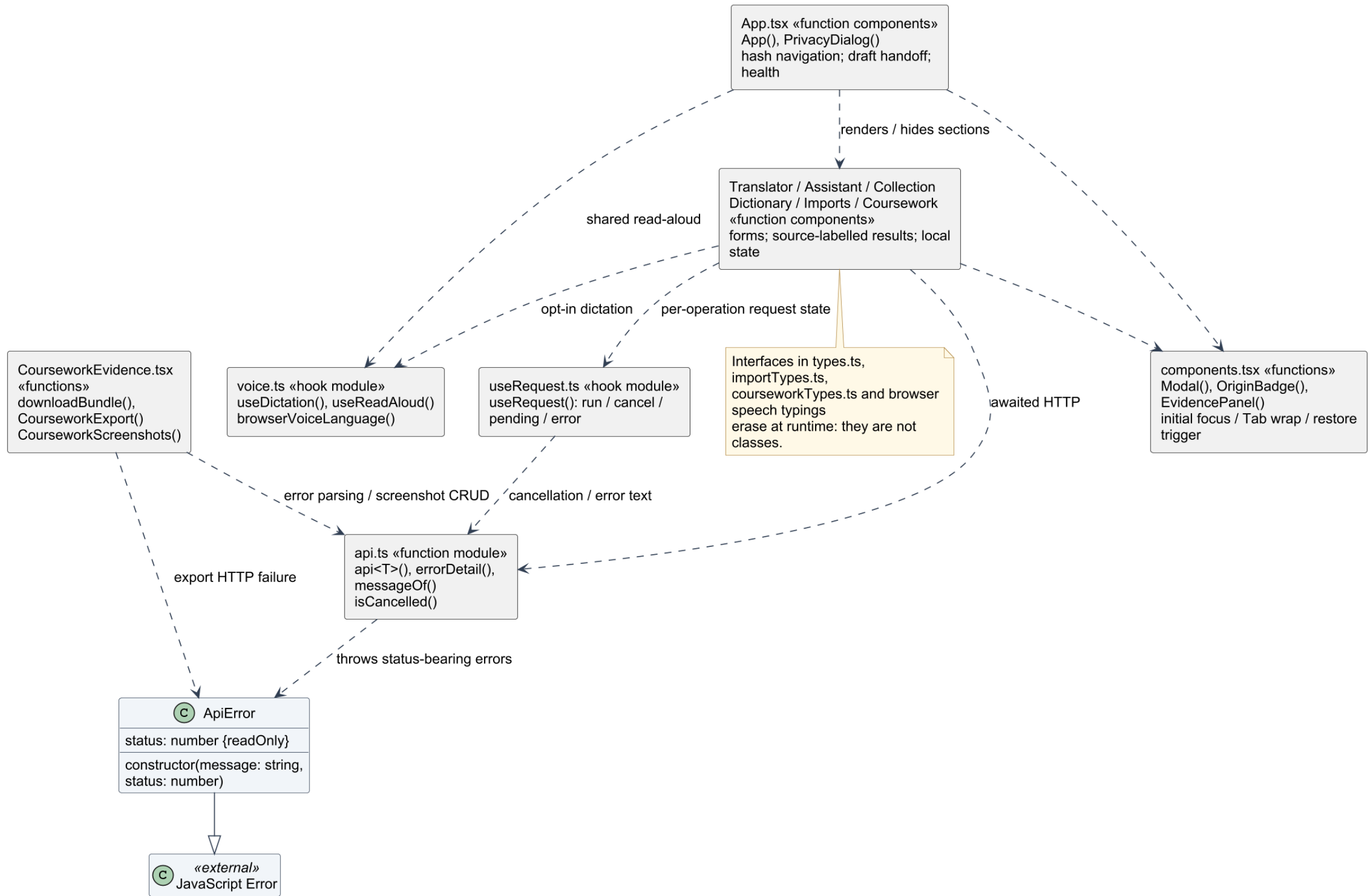
These classes are distinct from public `TranslationContent/ChatResponse` DTOs.

Actual provider service and local grounding result



GeminiService is a real class. Retrieval and route handlers are functions. Grounding is a value holder, not a hidden training or translation service. At most 8 selected evidence records are emitted by retrieve().

Frontend class and function-component structure

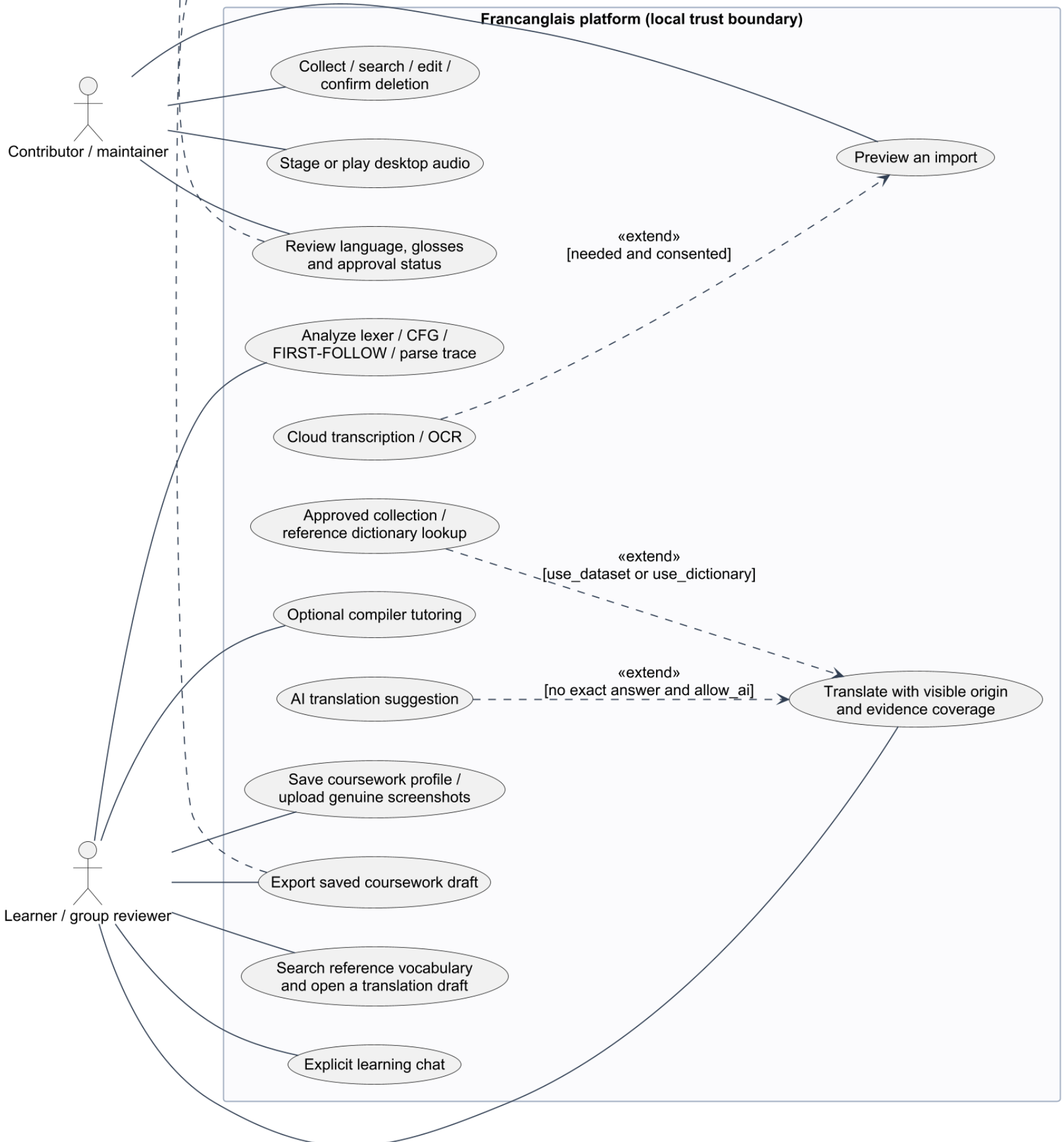


ApiError is the only authored production TypeScript class.
React function components and hooks must not be recast as service classes.
Module dependencies have no association multiplicities.

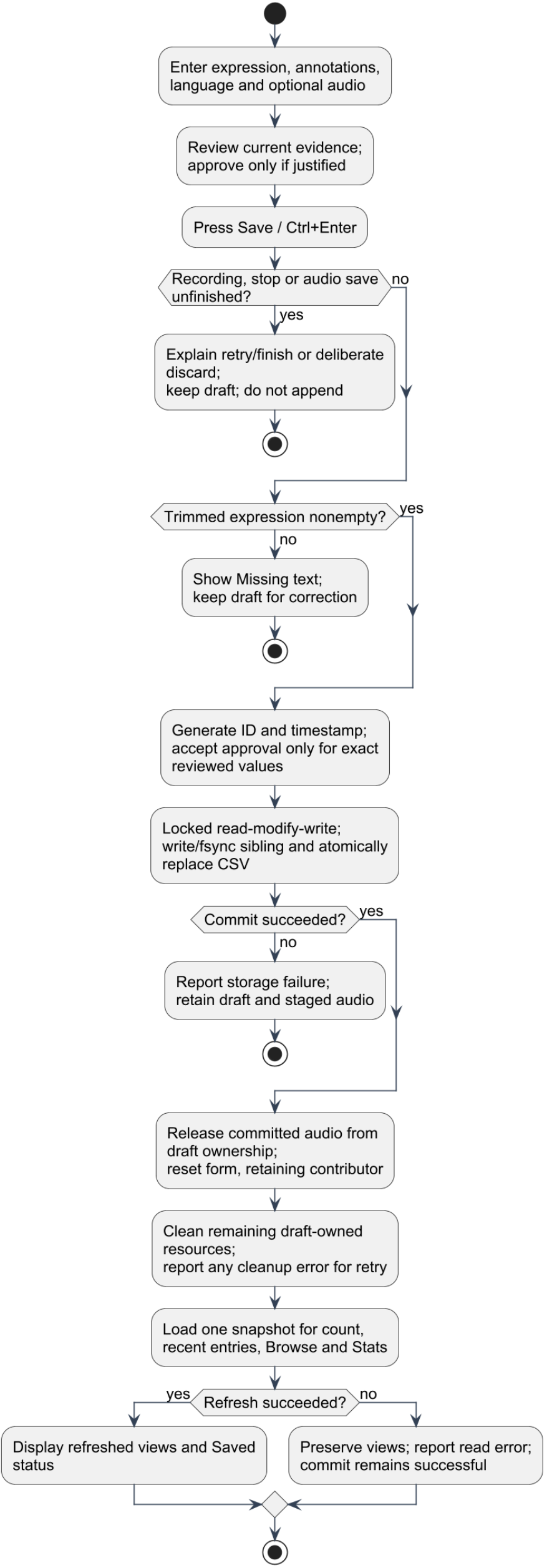
Implemented user goals and explicit optional processing

Draft generation is not completed fieldwork, final report pagination or group sign-off.

Saved does not mean approved. Preview/handoff never auto-saves. Roles are not authentication permissions. Dictionary search/handoff does not create fieldwork.

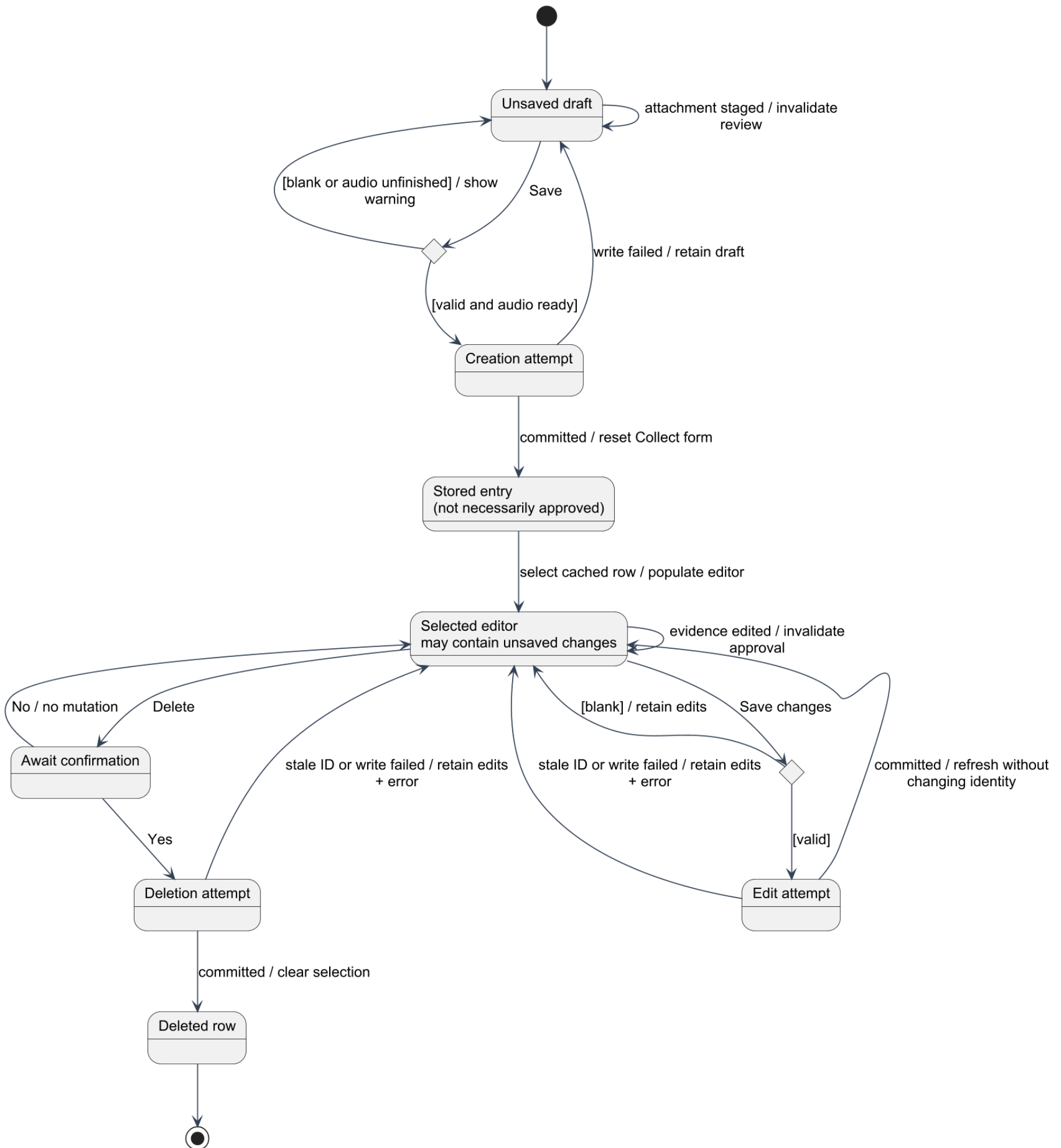


One desktop save attempt, including failure boundaries



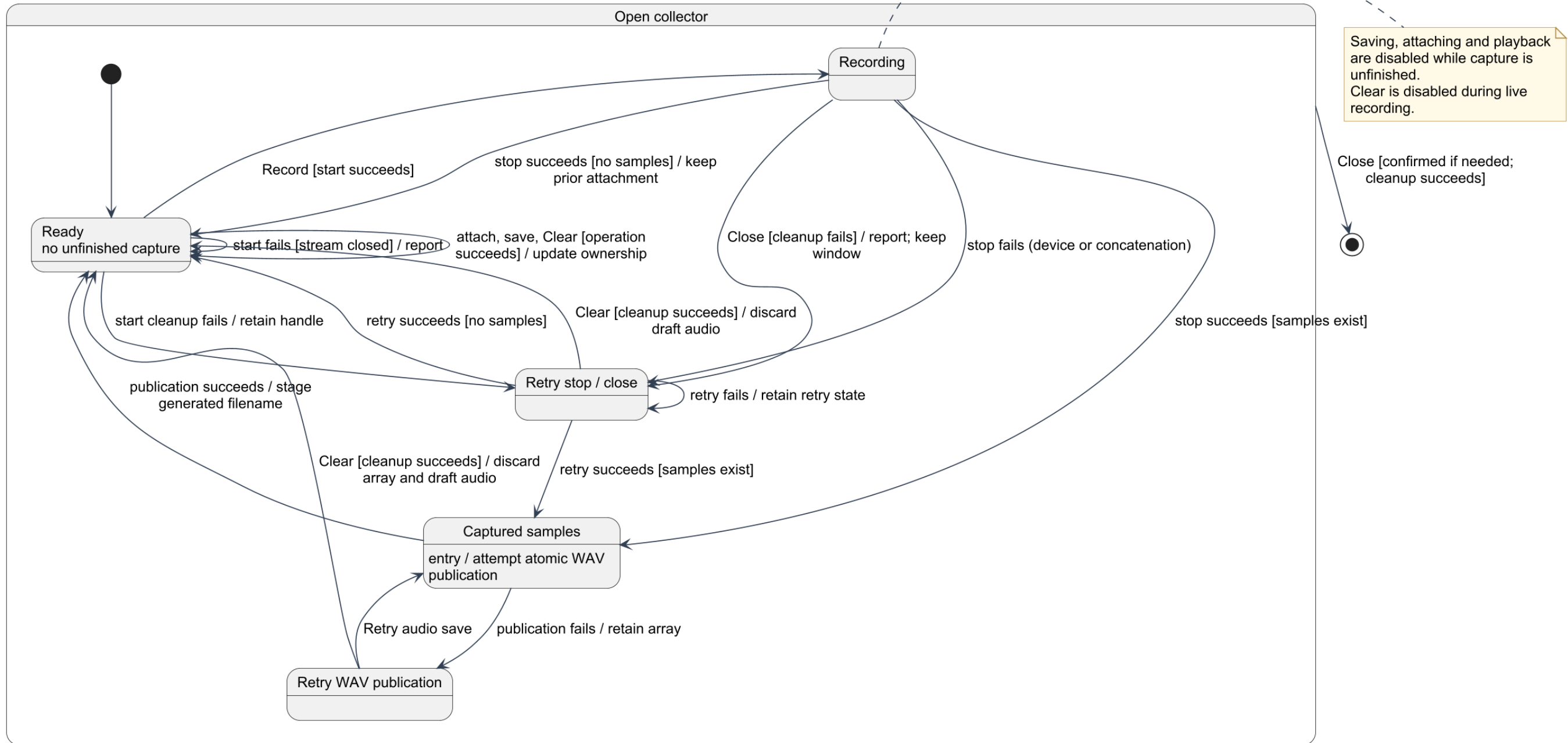
Diamonds represent exclusive guarded decisions, not parallel work.
One final node ends this attempt, not the collector application.
Duplicate warnings are advisory on desktop; API duplicates return 409.
File staging is separate; recording details appear in the sequence view.

Logical entry and selected-editor lifecycle



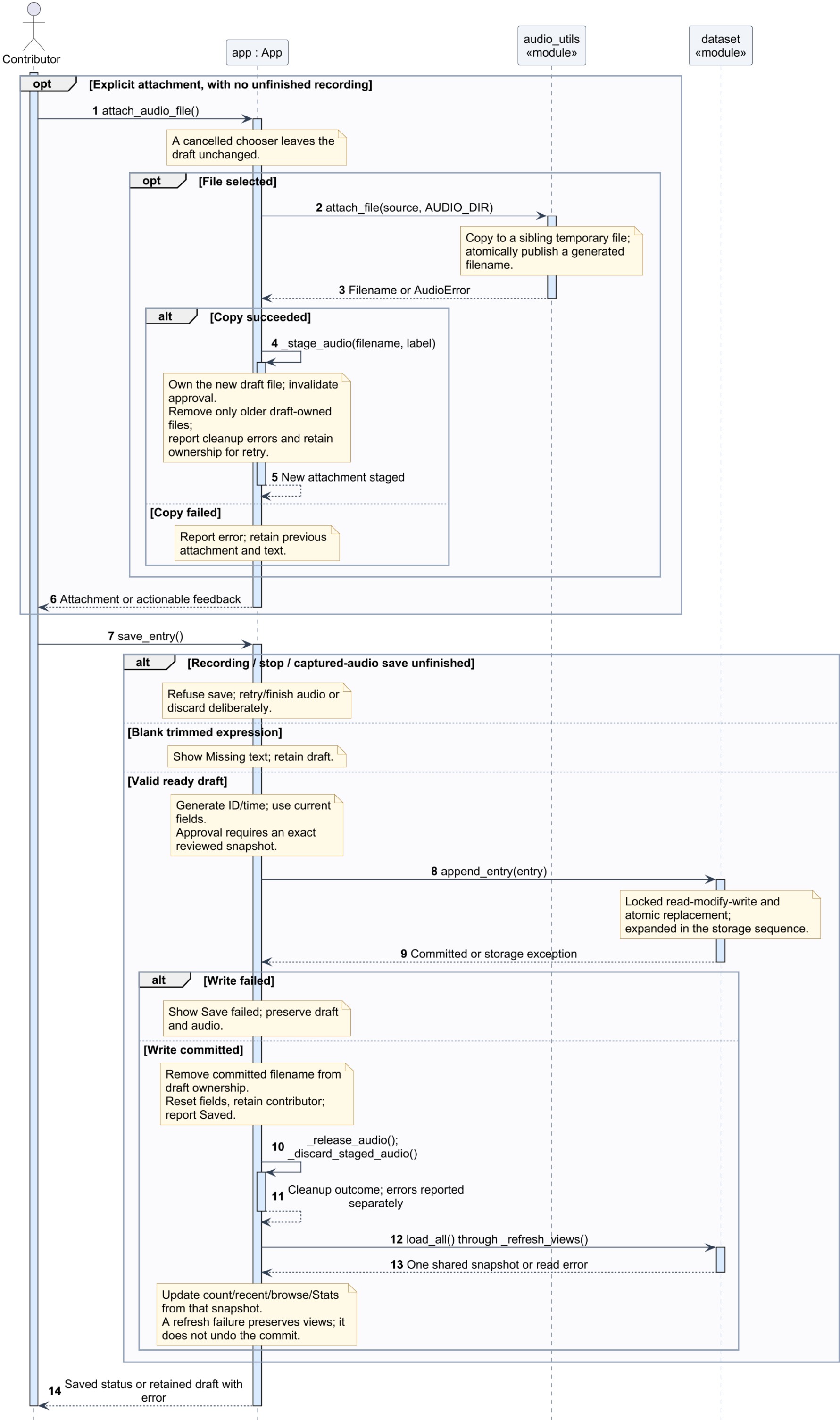
This is a logical row/editor model, not an implemented enum or whole-app lifecycle. The selected editor may outlive a row removed by another writer; stale errors are explicit. A successful commit is not reversed by later display/cleanup failure. Audio staging has a separate lifetime; deleting a saved row retains its recording.

Desktop capture readiness and retry states

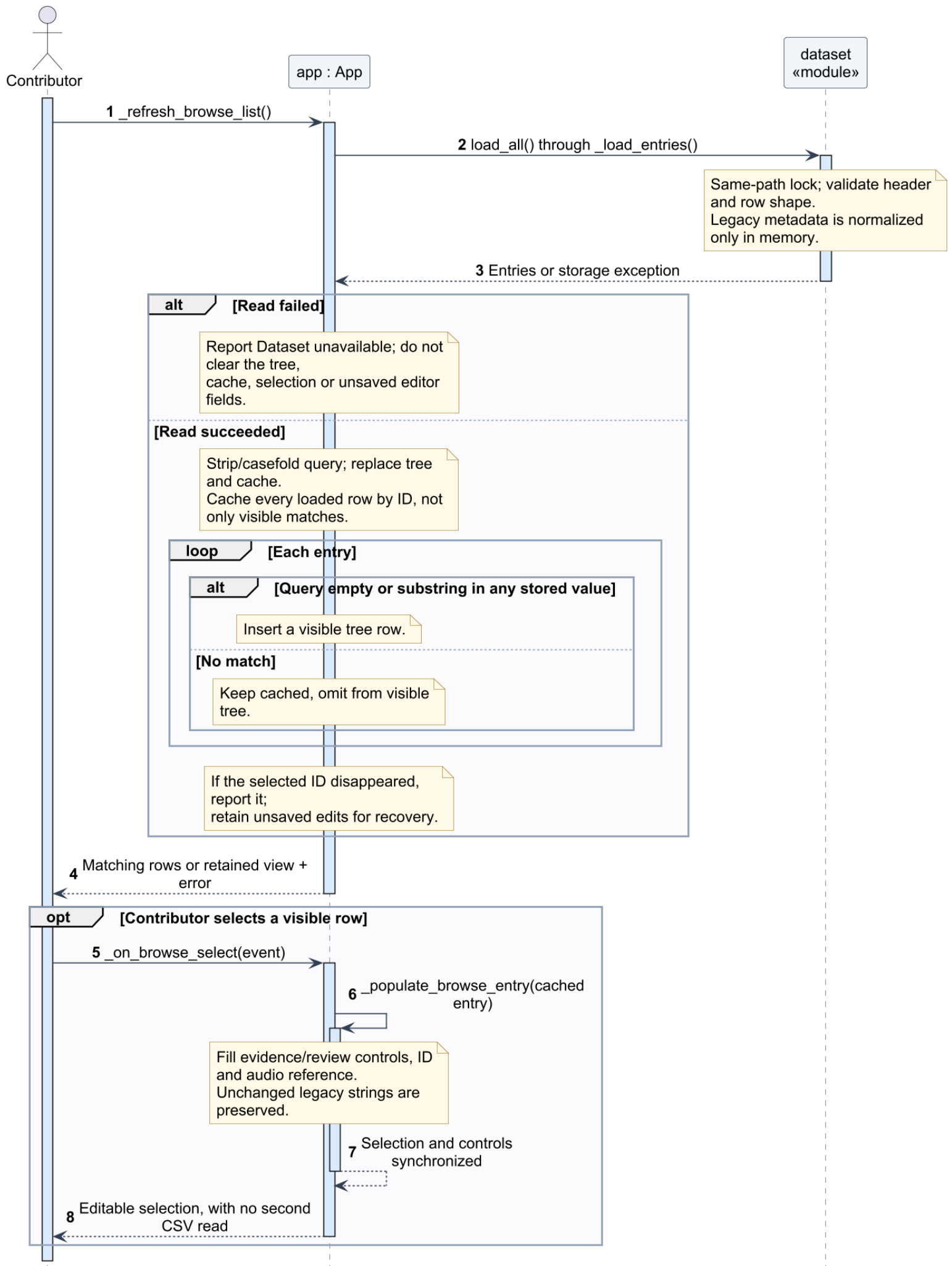


Conceptual states derive from recording, stream, pending-stop and captured-array fields. Ready may contain a staged filename; it does not mean an empty form. Failed cleanup keeps the window/retry state available, not a false closed status. Draft-owned files may be discarded; committed recordings are never owned by the draft.

Desktop attachment and truthful entry saving

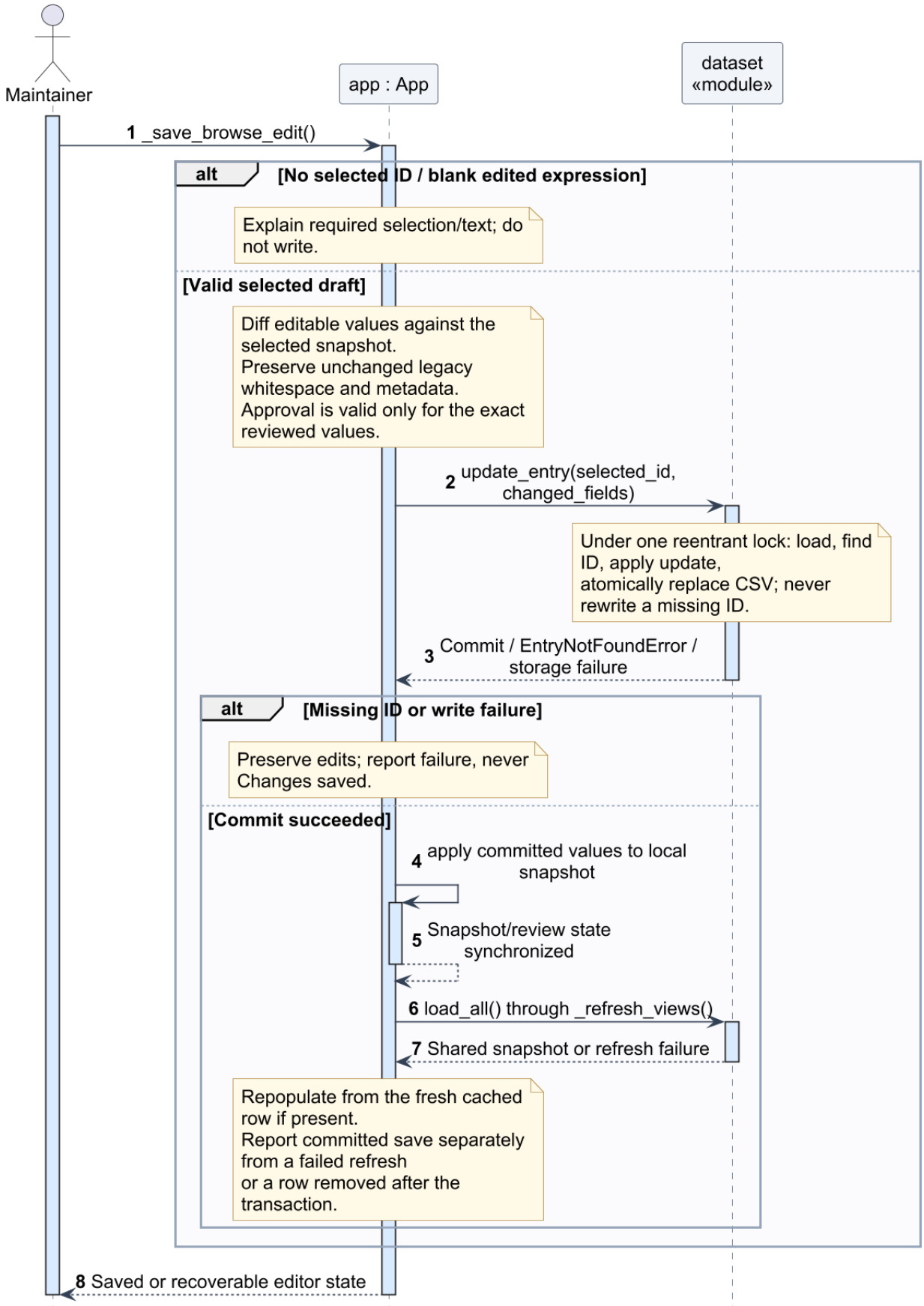


Desktop search, preserved views and cached selection



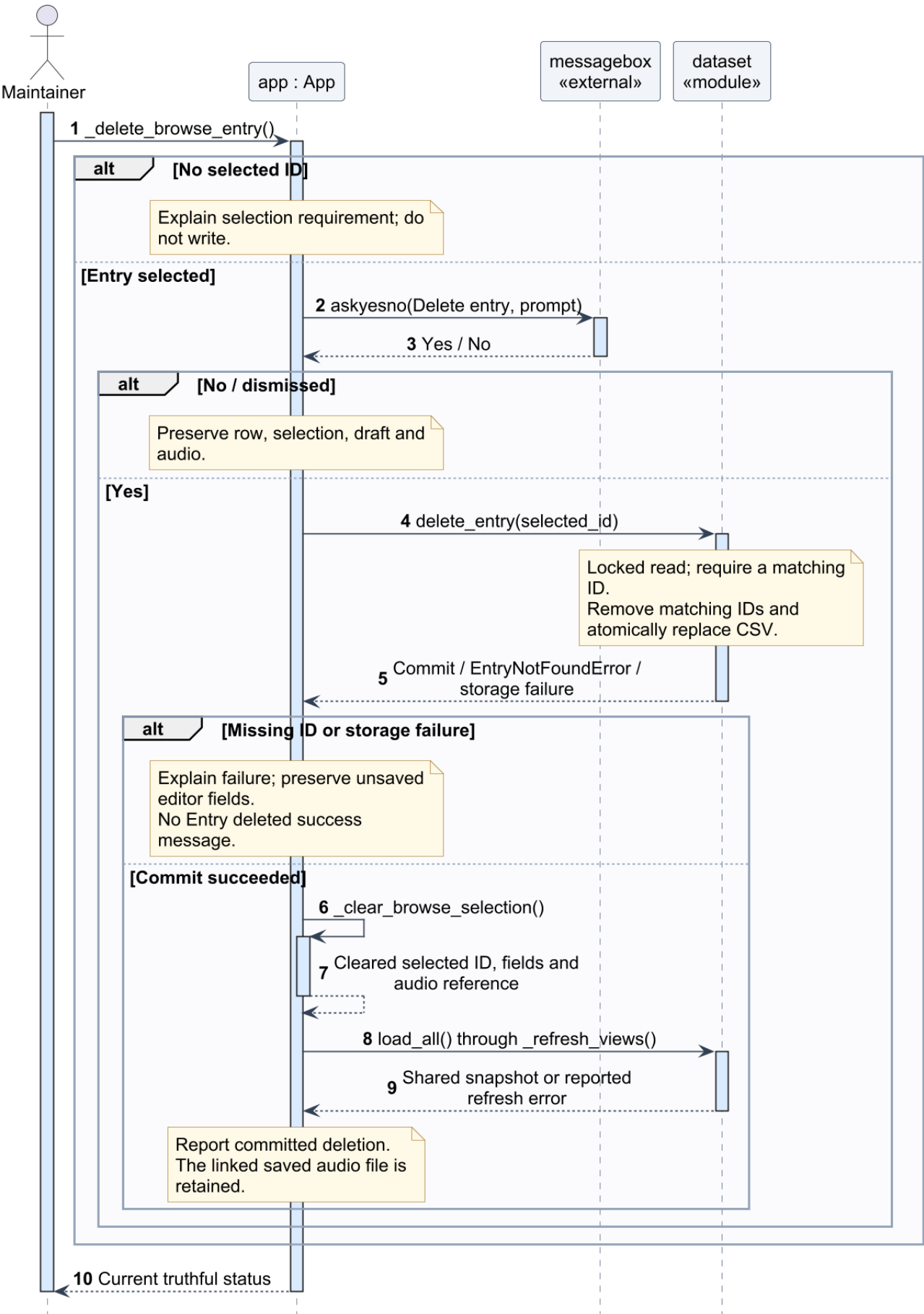
IDs should remain unique; manually corrupted identifiers are not repaired here. Filtering is read-only. Empty results are distinct from a failed read.

Desktop changed-field editing and review integrity



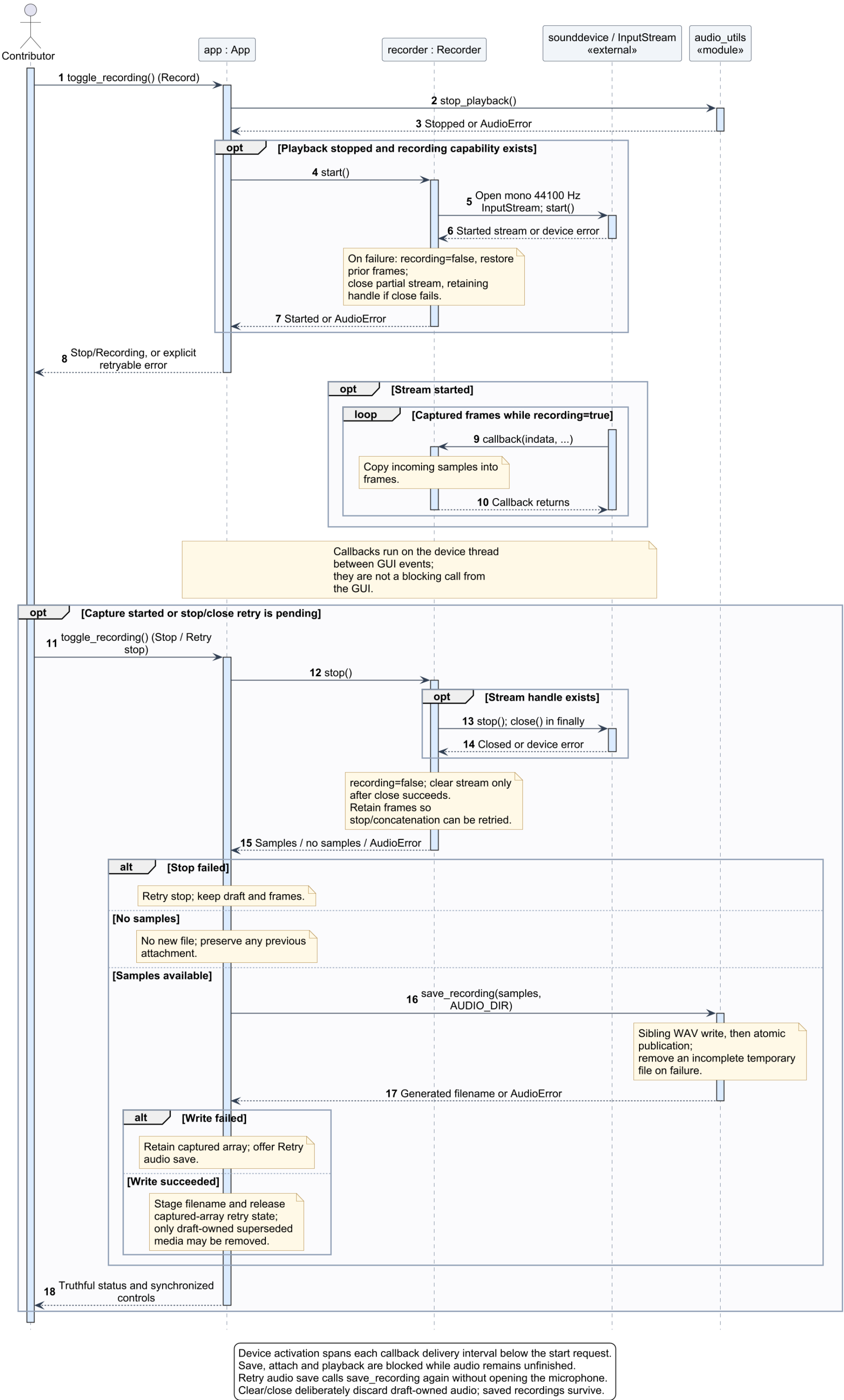
Desktop edits preserve ID, timestamp, contributor and audio reference. Text, glosses, topic, location, notes, language and lexical metadata are editable. Locks prevent interleaved writes, not semantic last-writer-wins conflicts.

Confirmed deletion without false success or media loss

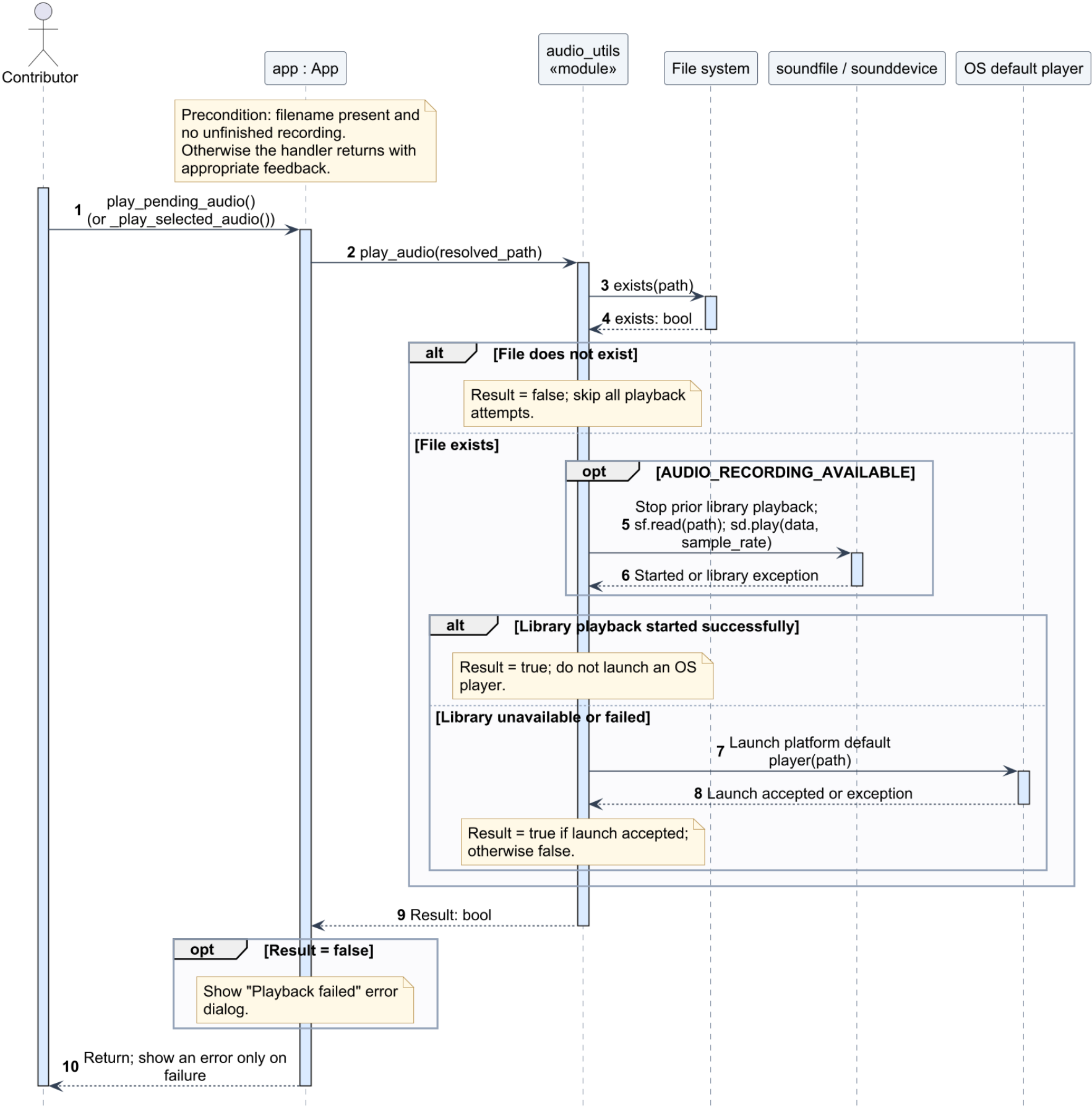


Confirmation is a decision, not an authorization or authentication check.
Atomic replacement protects write integrity; there is no undo stack.

Retryable microphone capture and WAV staging

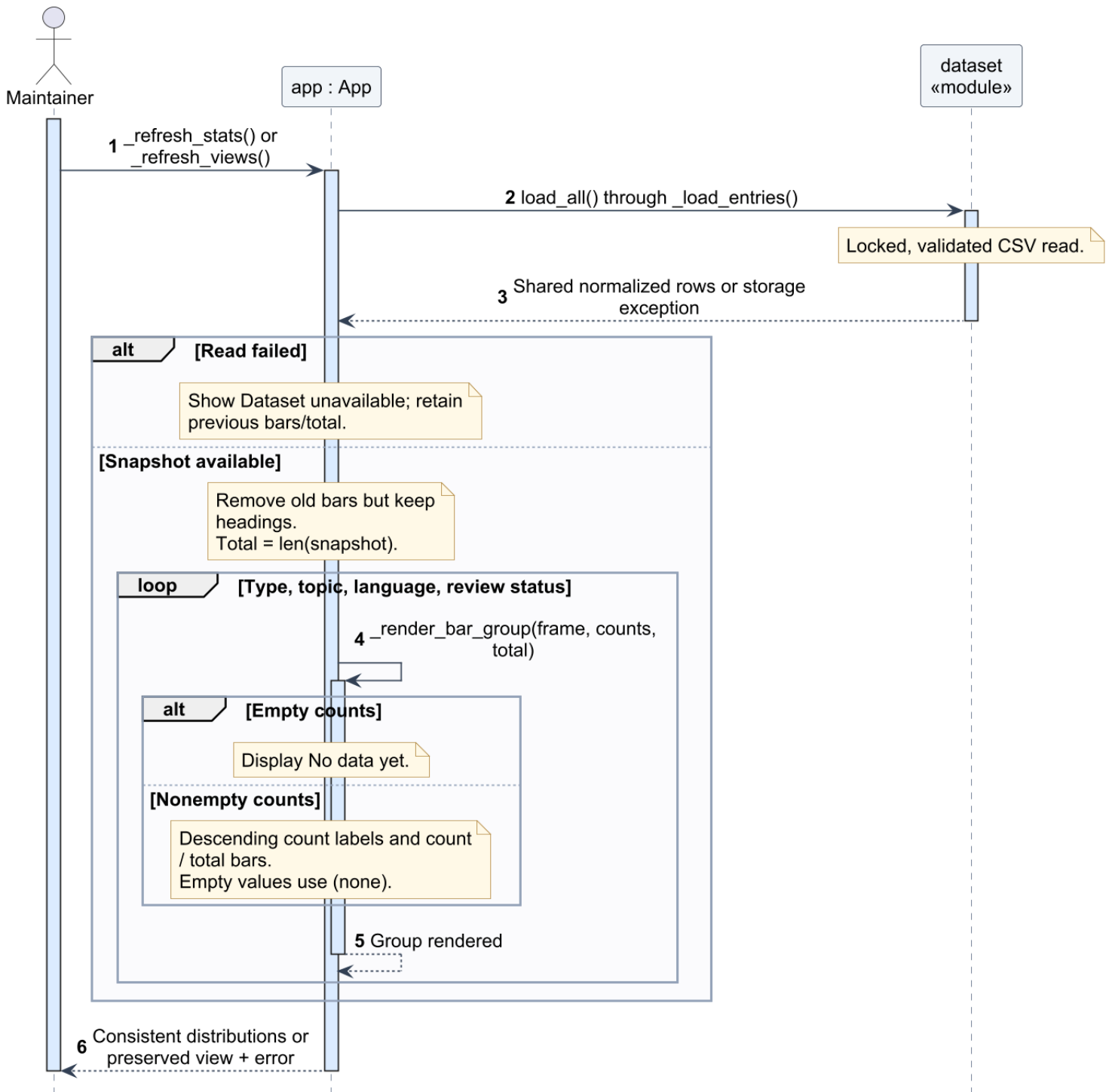


Play Audio with Library and OS Fallback



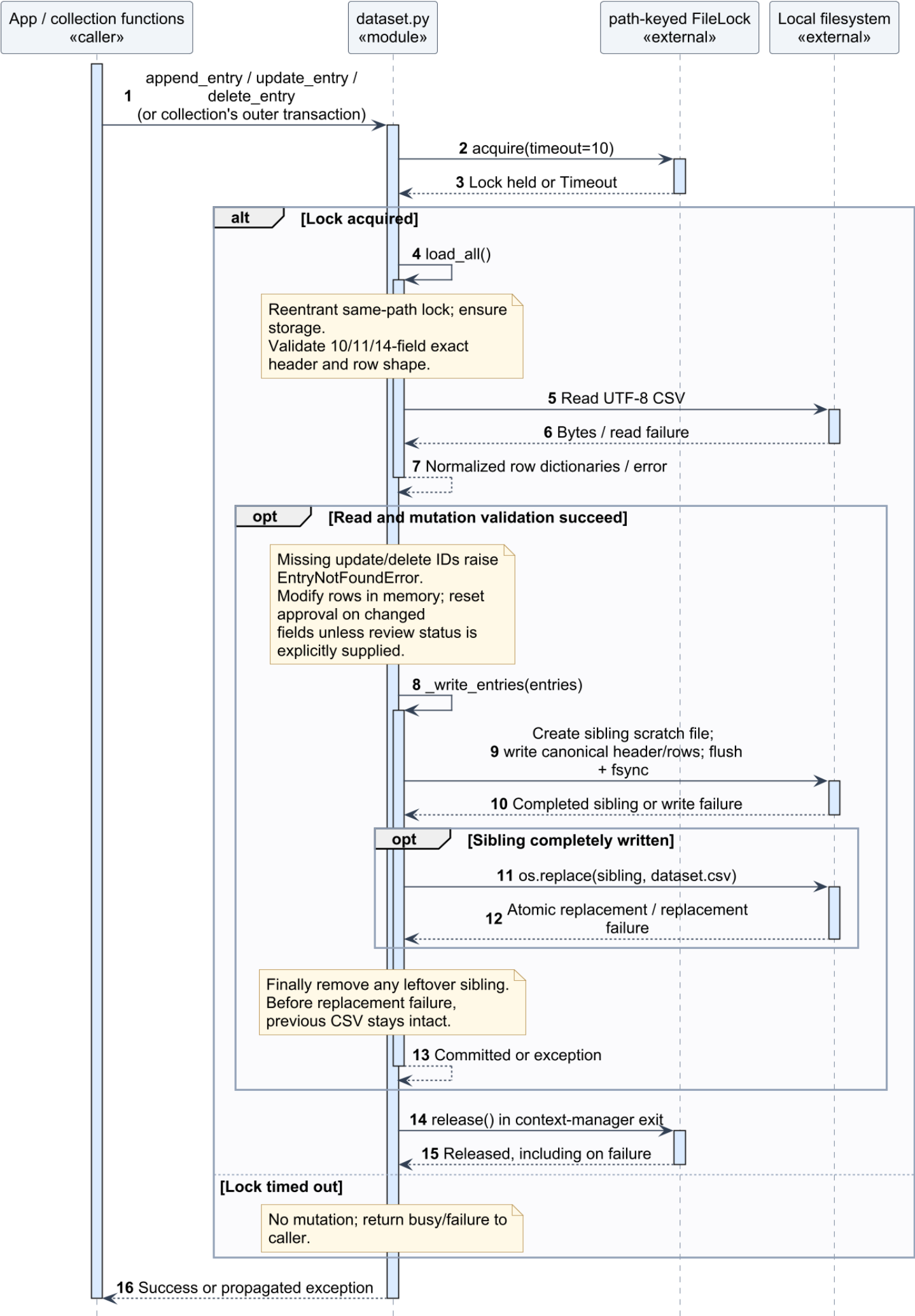
true means a playback request was accepted, not that sound was heard to completion. OS launch acceptance is not playback completion; media decoding is external. Expected audio errors are reported by the GUI; CSV is unchanged. Clear/close can stop in-process playback, not an external OS player's lifetime.

Consistent four-way statistics from one snapshot



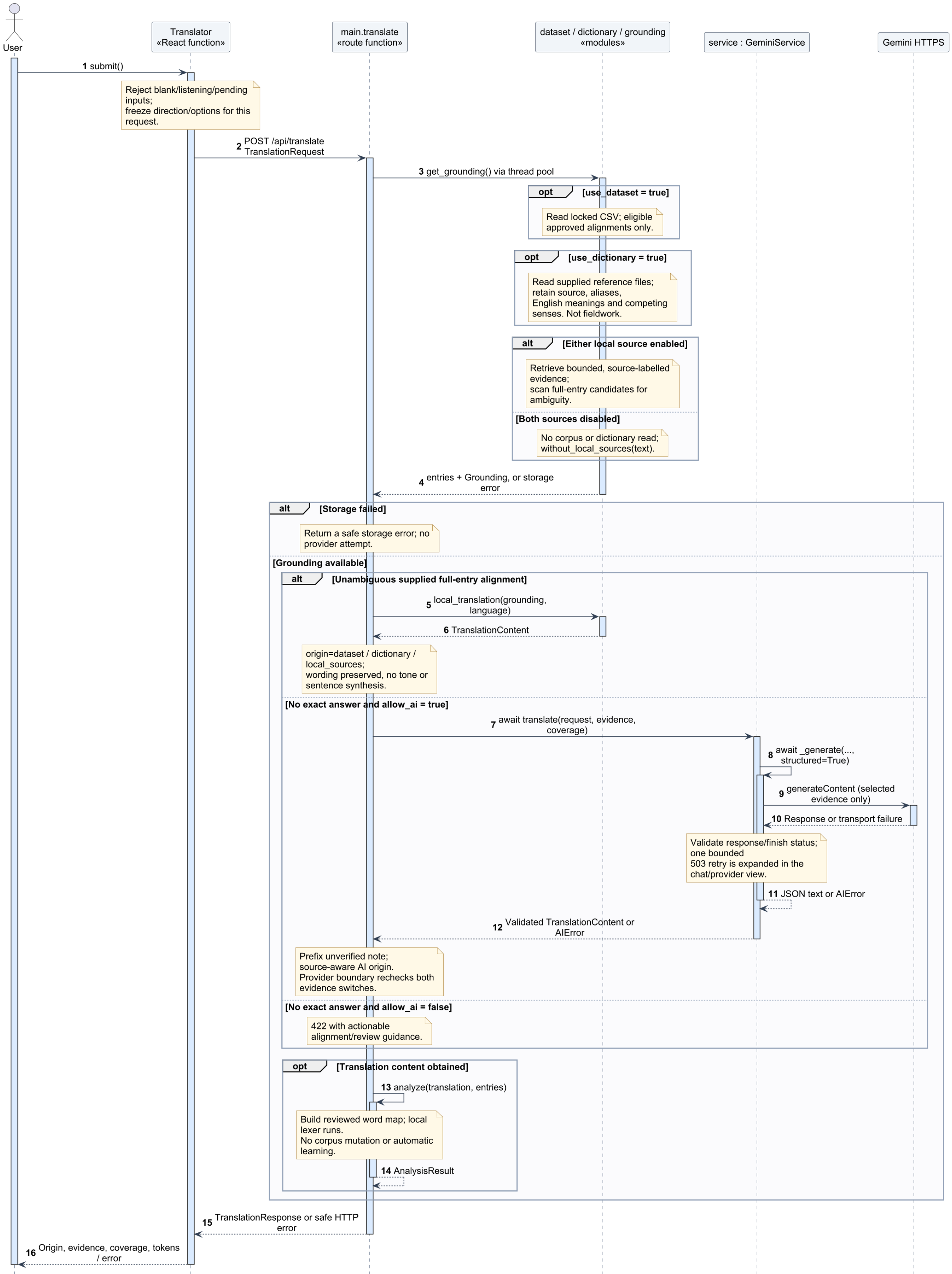
`_refresh_views` also supplies the same rows to count, recent and Browse views. Direct Stats refresh loads once; `count_by()` is not called by this handler. Performance evidence includes active-tab Tk idle rendering, not only CSV timing.

Shared locked CSV mutation and failure-safe replacement



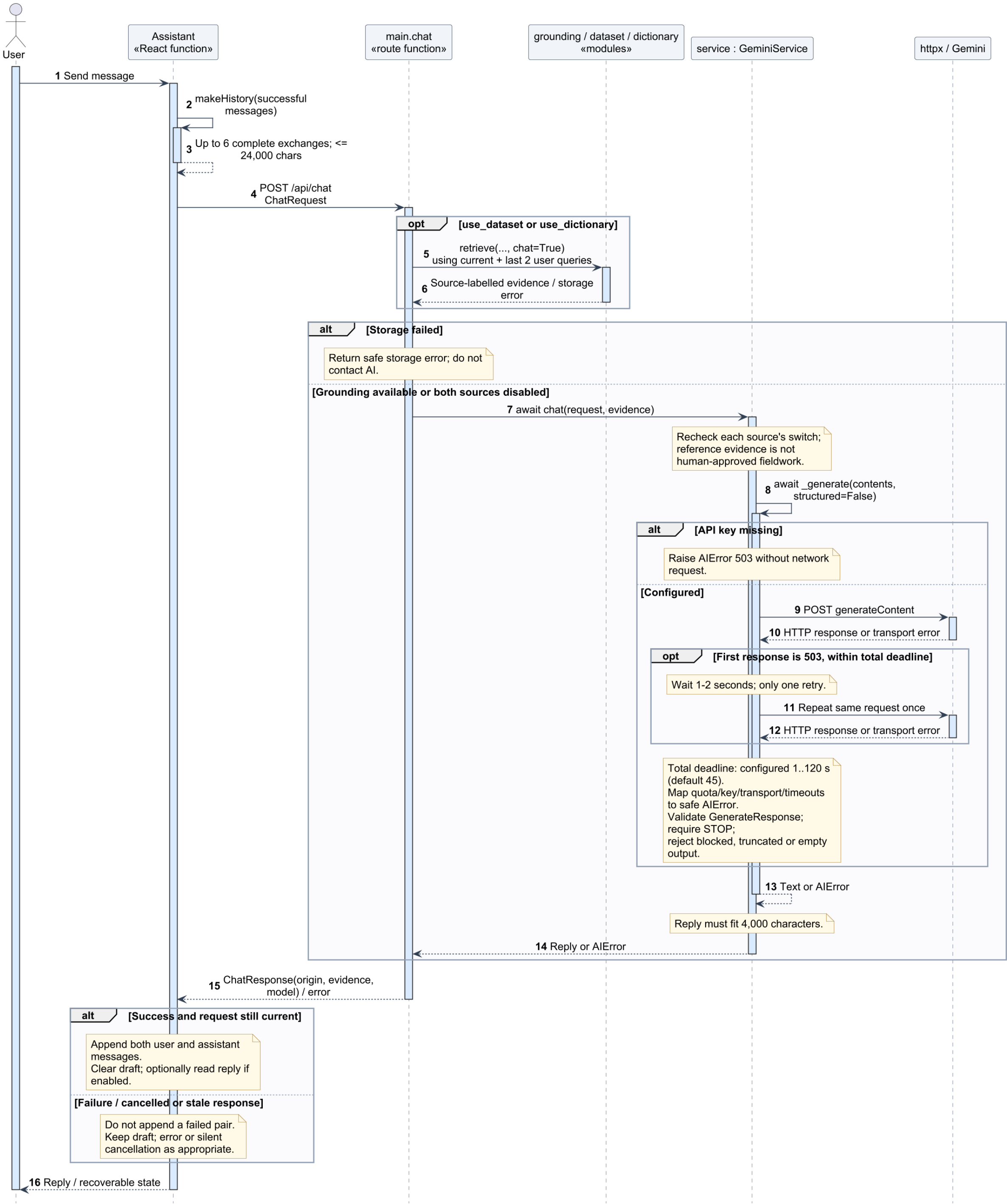
collection functions take the same outer lock around duplicate/ID checks and write. Lock activation shows the acquire/release call, not the duration of file ownership. Append also rewrites the full dataset; there is no unsafe direct append path. Protection covers cooperative users of this path, not external manual writers.

Web translation: enabled local sources, then optional AI



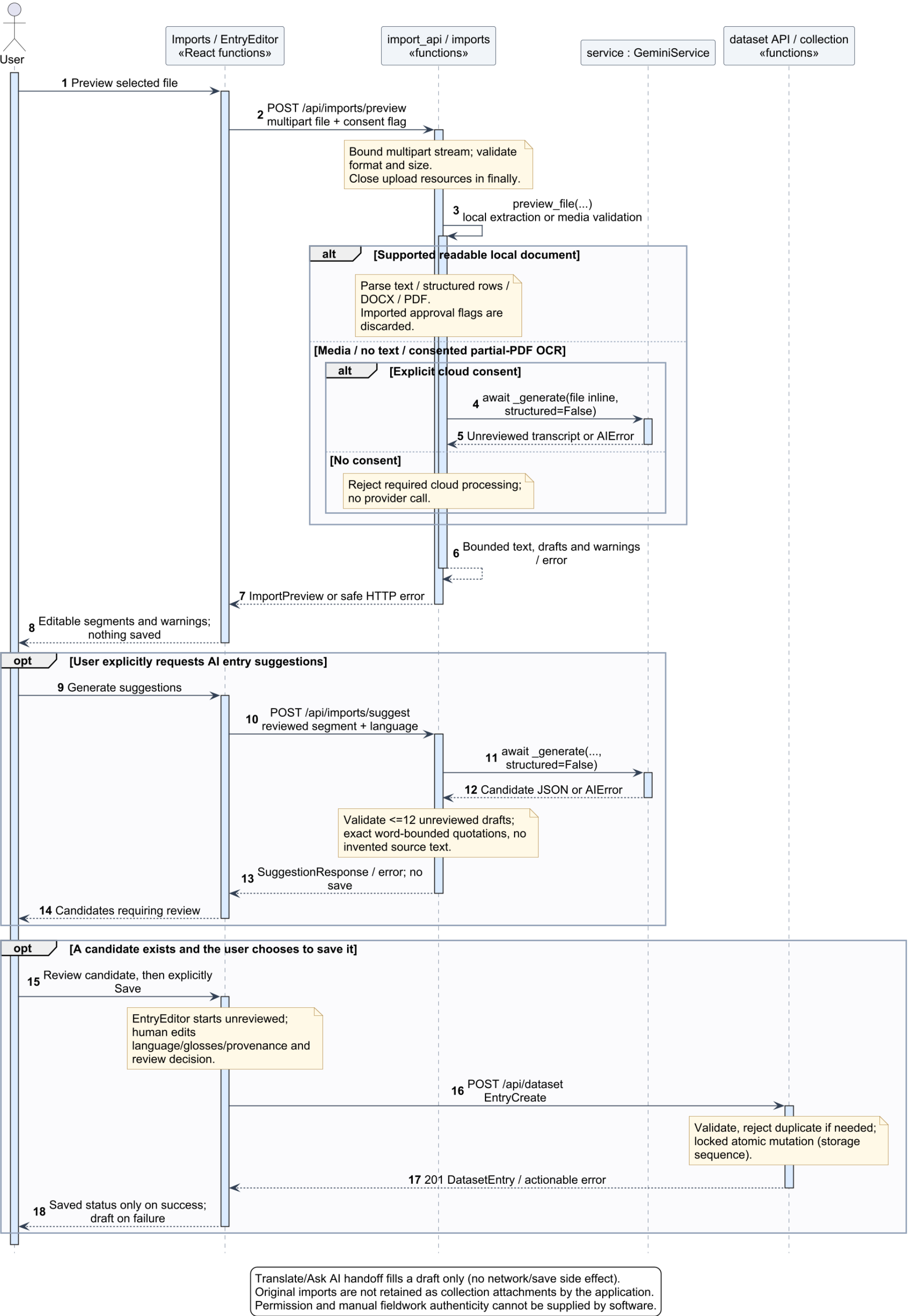
Awaited work is shown as a request/reply interval, not a blocked UI thread. Storage/validation/provider failure exits to an error; it never becomes a success. Cancellation prevents stale display, not guaranteed server/provider rollback.

Explicit chat and bounded provider response handling

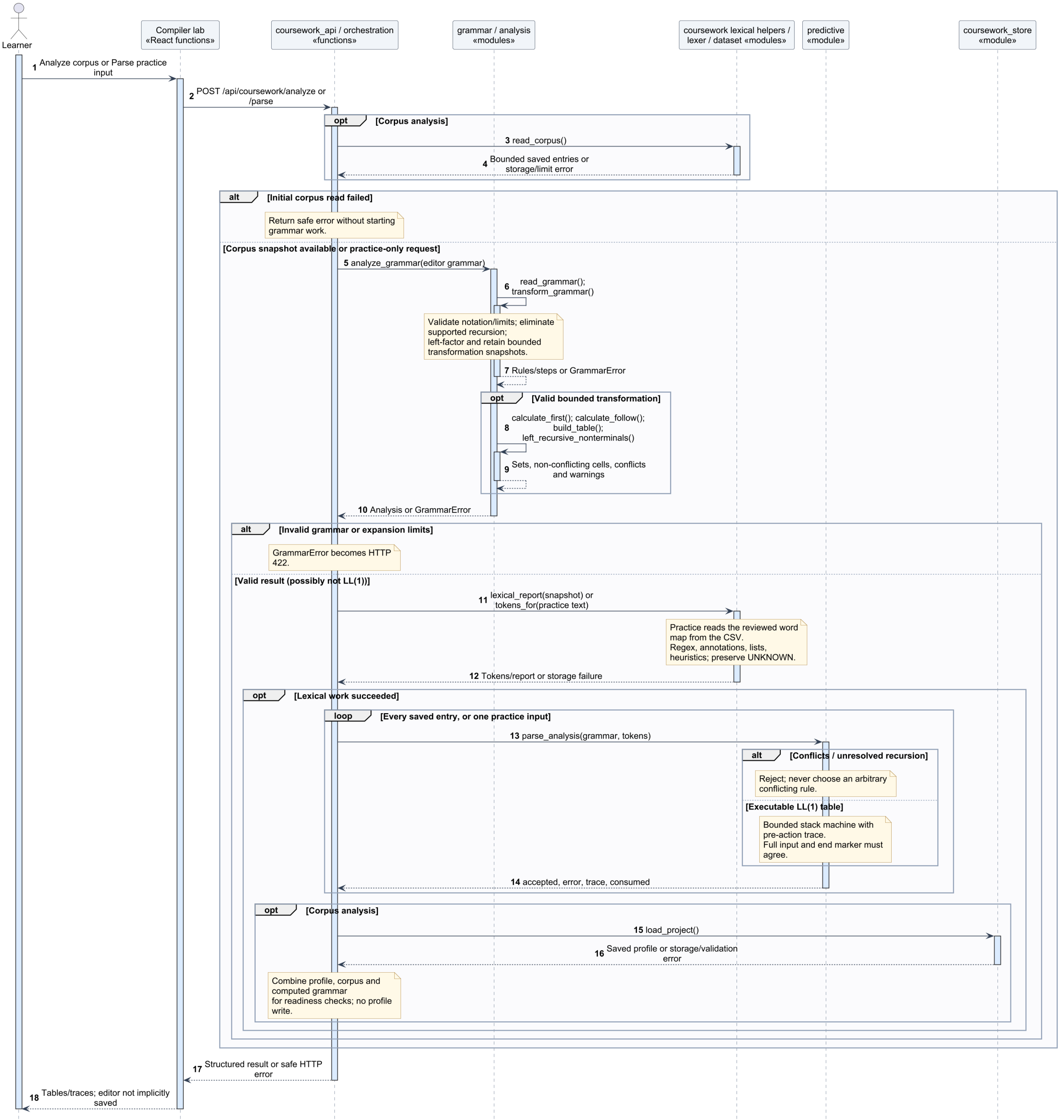


The collection and dictionary switches are independent.
Disabled sources supply no retrieved examples; typed text/history is still sent.
The model never receives contributor/location/notes through retrieval.
Clear removes browser-tab history, not possible provider-retained records.

Import preview, explicit suggestions and human review/save

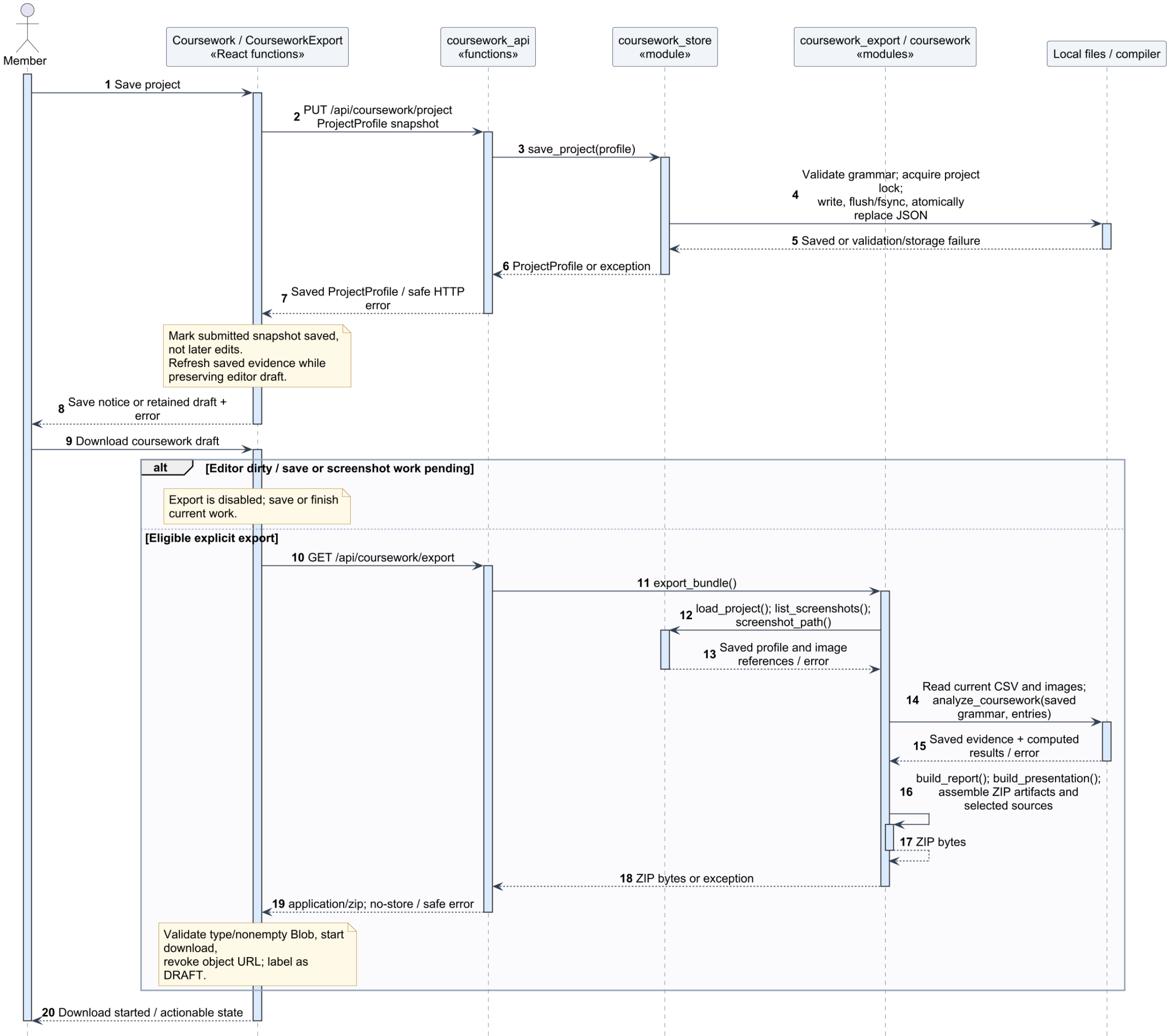


Deterministic grammar analysis and category-token parsing



Module self-calls condense internal function boundaries.
No provider determines parsing. Corpus/grammar fit is not linguistic authenticity.
Existing corpus/media stay unchanged; absent storage can bootstrap an empty header.

Save coursework, then export only saved evidence



No unsaved practice input or AI answer is included.
ZIP includes actual corpus/provenance/profile: review before sharing.
Individual files are protected, but export is not a global multi-file transaction.
Missing research remains visible; 25 sections do not guarantee 25-30 final pages.

Reference lookup: separate sources, explicit draft handoff

